Synthesizing symbolic models of human and machine reasoning at scale

Solim LeGris[1,2*], Philipp Mondorf[3], Jinran Jin[4], Akshay K. Jagadish[5], Brenden M. Lake[2,4], Todd M. Gureckis[1,6]

[1*]Department of Psychology, New York University.
[2]Department of Psychology, Princeton University.
[3]Center for Information and Language Processing, LMU Munich.
[4]Department of Computer Science, Princeton University.
[5]Princeton AI Lab, Princeton University.
[6]Center for Data Science, New York University.

*Corresponding author(s). E-mail(s): solim.legris@nyu.edu;

Abstract

While symbolic modeling has historically been limited to more restricted experimental domains, this approach remains useful for explaining cognitive representations. Here, we introduce a method for scaling symbolic model synthesis toward open-ended problem-solving, and apply our method across hundreds of problems in a domain that is challenging for both people and machines. Focusing on incorrect responses, we reverse-engineer interpretable, deterministic programs that explain a large proportion of human and reasoning language model inferences. By analyzing these programs, we find that both people and machines exhibit systematic patterns of simplification, find rules that are partial explanations for their observations, and produce structured cognitive errors which we characterize. While our results suggest that reasoning models often infer structurally different rules than people, machine-based analyses suggest that these shared signatures are modulated by an allocation of resources, with reasoning effort predicting observed trends. Together, these results provide bottom-up, program-level cognitive footprints of shared underlying mechanisms reminiscent of satisficing in both people and machines, and our method offers a general approach for comparing open-ended problem-solving behavior of black-box systems—whether natural or artificial.

Symbolic models have provided an immensely productive framework for modeling human cognition [1–10]. However, the recent successes of large language models (LLMs) [11–13] have redirected focus toward strongly predictive models within cognitive science [14–20], while purely symbolic models have lost traction [21]. This is partly due to constraints arising from the extensive hand-engineering required to fit symbolic models. For instance, the primitives from which hypotheses are constructed must be posited in advance [22], as well as the form those hypotheses may take—typically a grammar over admissible compositions [5, 10, 23–26]. Further, for individual-level predictions, modelers often handcraft a limited collection of possible strategies or deviations [27–30]. In restricted experimental settings these aspects of cognitive modeling are tractable, but in more open-ended domains they are not. Within large hypothesis spaces, neither the representations a cognizer may entertain, nor the many possible deviations from normative models can be enumerated in advance or efficiently searched due to issues of combinatorial explosion.

Here, we leverage recent advances in artificial intelligence (AI) to synthesize symbolic models at scale in a domain that is challenging for both people and machines. In particular, we exploit the emergence of strong programming abilities in LLMs [31–33] to address the issue of combinatorial search within discrete program spaces. We developed a pipeline for behavior-conditioned symbolic model synthesis which uses an LLM backbone to automate search over program representations of rules that people or machines may have entertained while solving problems. Specifically, we focus on modeling incorrect human and machine responses. Although correct responses in complex cognitive domains are the basis for rich inferences about cognition when paired with additional behavioral measures, *incorrect* responses can also serve as a rich window into underlying cognitive processes—a notion that guides our focus here on errors. While prior work has made important progress in automating cognitive model discovery [34–40], our approach is scalable to arbitrarily large collections of tasks sharing an underlying grammar, and recovers auditable individual-level collections of deterministic programs, rather than within-program aggregate parameter statistics. The resulting distribution of reverse-engineered programs allows us to account for patterns of generalization, expose representational biases, perform human-machine comparisons, and provide bottom-up insights into cognitive mechanisms supporting reasoning.

Nearly half of incorrect human responses we aimed to model are captured using our proposed procedure, and a similar proportion of LLM errors (43.1–54.2% across model configurations). Analysis of error programs suggests that people infer systematically simpler rules than the target rule for a given task, that their rules often explain ordered subsets of observed examples, and finally that people often incorrectly rely on simpler or more perceptually salient feature dependencies. Machine evaluations showed similar trends in simplification, partial-rule generation, and error type distribution, with distinctive but graded differences driven by model capacity, reasoning effort and structural differences between synthesized programs. Beyond recovering classic signatures of human inference in a domain that far exceeds the complexity of the experimental tasks in which such regularities are typically studied [41–44], we expose similarities and differences in abstract reasoning between people and machines using a shared symbolic representational space.

Behavior-conditioned symbolic model synthesis

We apply our method to the Abstraction and Reasoning Corpus (ARC), an open-ended domain of abstract rule induction tasks spanning a wide range of concepts [45], whose solutions can nonetheless be expressed within a single domain-specific language (DSL). The four subsets of participant-level response examples in Figure 1A illustrate that incorrect responses in this domain often reflect systematic patterns of inference rather than random or defaulting behavior. Thus, similarly to prior work [28, 29], we posited that such responses should correspond to constrained deviations from a normative standard, and assume that rule-like inferences of people and machines alike are best understood as programs. Here, we operationalize this using programs manipulating grids and implemented in a DSL. The DSL we leverage provides human-written reference programs for the 400 `training` tasks of ARC-AGI-1 [46]. Although our procedure provides a symbolic representation of the underlying algorithm a person or AI system may have applied to produce their response, it is not by itself a process-level description of how the program was inferred. In favor of scalability, we remain agnostic to what process may have generated such programs, and instead focus on characterizing the program that was applied to produce a response, with the goal of recovering process-level signatures. Specifically, for a given individual response, our method searches for programs representing output behavior by sampling constrained sets of deviations from a reference program. As illustrated in Figure 1B, given a reference program P_i^* implementing the target rule for a task T_i, an LLM proposes k independent sets of at most j line edits $E_1, \ldots, E_k$ to that program. Importantly, proposal sets are sampled conditioned on single human or machine behavioral responses. Each resulting candidate program $E_l(P_i^*)$ is then executed and retained only if it reproduces the response the proposal LLM was conditioned on. Thus, the LLM guides search over a discrete program space by supplying a proposal distribution for plausible deviations implemented as line edits, while program execution provides a hard verification mechanism.

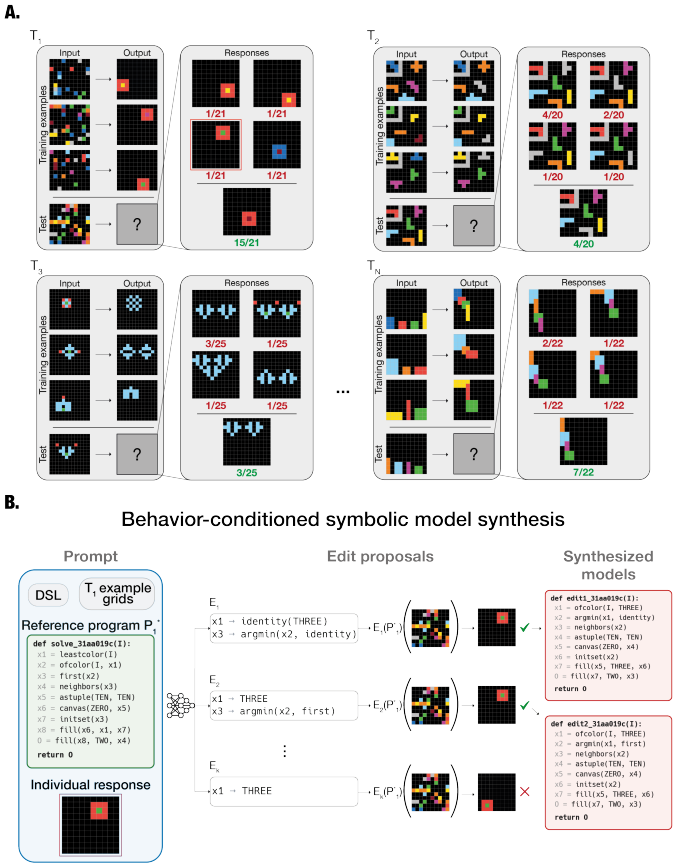


Fig. 1 Reverse-engineering individual rules in a complex cognitive domain using programs. (**A**): Task corpus and associated behavioral data. Each of the four representative ARC tasks T_i presents an agent with a small set of input–output training examples from which a hidden transformation rule must be inferred. From a test input, the agent must then construct the corresponding output grid (hidden with a grey square and "?" for illustrative purposes). We use H-ARC, a dataset of responses from over 1,700 participants to the 800 publicly available ARC-AGI-1 tasks. On the right of each task, we show 4 of the most frequent incorrect responses, and the correct response at the bottom. Under each grid, we show the associated response frequency (red for incorrect, green for correct). (**B**): Synthesis pipeline. For each unique incorrect response, a large language model is prompted with a full specification of the domain-specific language, task grids, a reference program P_i^* implementing the target rule for task T_i, and the observed incorrect response (here, from a human participant). It proposes k independent sets $E_1, \ldots, E_k$ of at most j line edits to that program. The schematic edit proposals E_l show what the LLM proposes to apply to P_i^*. Each proposed set of edits is applied programmatically to the reference program to obtain a new program $E_l(P_i^*)$. A custom parser rewrites the programs such that only causally relevant code is retained (i.e., unreferenced lines of code are removed). Each new program $E_l(P_i^*)$ is then executed with the test input as argument. Programs whose output reproduces the observed response are kept (green checkmark); all others are discarded (red cross). Because a response is often reachable by more than one unique program, the procedure returns a collection of programs per response rather than a single model. The individual response shown here is also shown surrounded by a red square to the right of task T_1 in panel A. The constant `THREE` represents the color green.

1 Results

We draw on prior work by leveraging the H-ARC dataset, a large corpus of human responses from over 1,700 participants to the full set of 800 publicly available ARC-AGI-1 tasks [47]. H-ARC contains up to three attempts per task from each participant, with correct/incorrect feedback after each attempt. We filtered H-ARC to narrow down the dataset to responses more likely to arise from conceptual rather than procedural mistakes (§3.1 for more details). The final filtered H-ARC dataset contains 6,261 unique incorrect outputs across 723 tasks and 1,387 participants. For the 400 `evaluation` tasks which did not have a human-written solver, we successfully synthesized reference programs for over 90% of the tasks (see §3.2.1 for more details), and conditioned error program synthesis using those programs. Unless specified, we report human results aggregated over modeled attempts from the `training` and `evaluation` splits. Wherever substantial divergence was observed, we report results separately. To compare human and machine errors, we additionally evaluate a range of large reasoning models (LRMs) on the 400 `training` tasks of ARC-AGI-1 (see §1.5 for more details).

1.1 Synthesized programs capture human errors

After verification and filtering (see §3.3 for more details), we obtained 8,657 total programs matching individual responses to tasks in both the `training` and `evaluation` splits of ARC-AGI-1. These programs account for 52.5% of non-trivial incorrect attempts on the `training` split, and 46.2% on the `evaluation` split (49.1%, 3,636/7,402 pooled; Supplementary Table S2 for the full per-split breakdown). Given the edit constraints on the synthesizer, our results demonstrate that a large proportion of incorrect human responses can be modeled by programs a few line edits away (M=2.84, SD=2.04, §3.3) from reference programs implementing the target rule for each task. Further, among successfully modeled incorrect responses, we recover a median of 2 distinct programs per unique error grid (IQR: [1, 4]). This many-to-one correspondence is illustrated in Figure 1B with two distinct, but successful edit sets. The first replaces the `leastcolor(I)` primitive operation with a hardcoded color, while the second additionally substitutes a selection operator: both yield programs whose output reproduces the same participant response, and both are retained.

We compared our approach against an exhaustive search over all single-line rewrites of the `training` reference programs (§3.3). Despite enumerating over 16M edits, this baseline captures only 25.8% (899/3,485) of erroneous attempts—roughly half of what behavior-conditioned synthesis achieves on the same data. With only 15 synthesized samples per response, our approach still recovers 89% of the exhaustively searched programs which match human responses. Because the number of candidates grows combinatorially with edit depth, exhaustive search is intractable precisely at the edit distances where many human errors lie. We further validated the pipeline on 148 synthetic errors with known generating programs (§3.5). At an edit budget of $k = 3$, the pipeline recovered a program reproducing all but one validation grid (99.3%, up from 84.5% at $k = 1$).

Under the view of ARC as program induction, our results suggest that people often infer incorrect programs that are structurally similar to the program implementing the target rule for a given task, providing support for our initial hypothesis that incorrect responses in this open-ended domain are not merely idiosyncratic failures.

1.2 Inferred programs are systematically simpler

Human learners intrinsically value simple representations over more complex ones [41, 42, 48], and this fact is sometimes an essential component of models posited to capture human behavior [5]. Here, we examine whether this bias for simple codes is a property of the behavioral outputs which we modeled using our approach, beyond the shortening bias intrinsic to the edit procedure of our pipeline. As a test of this idea, we applied the same procedure used for producing error programs, but instead conditioned the synthesizer on *correct* output grids (§3.8 for more details). This measures the inherent tendency of our approach to produce shorter programs independent of the nature of the errors. Because the programs are adapted using line-level edits, we use relative line-count change as a proxy for change in program complexity with respect to the reference programs. Statistical analyses confirm that DSL programs that capture human responses are credibly simpler than reference programs they are adapted from, on both splits (Figure 2C). On the `training` split, where each reference program is hand-written by a human, error programs are 19.1% shorter on average ($\beta = -0.191$, 94% HDI $[-0.206, -0.176]$, $P(< 0) = 1.00$), while baseline (correct-output) programs are only 5.2% shorter ($\beta = -0.052$, 94% HDI $[-0.076, -0.028]$). This corresponds to an additional shortening of 13.9 percentage points attributable to conditioning on human errors ($\beta = -0.139$, 94% HDI $[-0.163, -0.114]$, $P(< 0) = 1.00$). The same pattern holds on the `evaluation` split, with the caveat that its reference programs are also model-synthesized rather than hand-written. Error programs from the `evaluation` split are 24.3% shorter on average ($\beta = -0.243$, 94% HDI $[-0.259, -0.225]$, $P(< 0) = 1.00$) versus 2.7% for baseline programs ($\beta = -0.027$, 94% HDI $[-0.055, -0.002]$), an additional 21.5 percentage points attributable to conditioning on human errors ($\beta = -0.215$, 94% HDI $[-0.240, -0.189]$, $P(< 0) = 1.00$). Panel D of Figure 2 illustrate a drastic but intuitive example of simplification: the target rule for this task requires searching over the eight rotations and reflections of a framed sub-grid for the one consistent with the surrounding cells, but the programs reproducing the two bottom responses instead use fixed transforms (`hmirror` and `rot180`, respectively), rendering fourteen lines of the reference program irrelevant to the final output.

To examine this effect further, we performed statistical analyses of relative line-count change in error programs as a function of reference program line count, empirical task success rate and error categories for `training` split tasks only (discussed further in §1.4). Our statistical model finds that reference program complexity modulates the magnitude of relative shortening. For each additional line in a reference program, we find 0.47 percentage points more shortening on average in corresponding error programs ($\beta = -0.0047$, 94% $[-0.0056, -0.0037], P(< 0) = 1.00$). For example, error programs derived from a 7-line reference program are predicted to be 5.8% shorter on average, while those from a 28-line reference are 15.7% shorter. This corresponds to

a 9.8 percentage-point difference for a four-fold increase in reference program length (Supplementary Figure S3A).

Additionally, we found suggestive evidence that conditional on a reference program's line count, errors from tasks with higher empirical success rate are modeled by slightly less relative shortening ($\beta = 0.0227$, 94% HDI $[-0.0213, 0.0666], P(> 0) = 0.84$). The estimated coefficient suggests that easier tasks tend to elicit error rules slightly closer to the reference programs in complexity, but the credible interval includes zero so these results remain inconclusive.

Although the procedure is biased toward shortening, our analysis suggests that, beyond this bias, the incorrect rules that people infer are often represented by substantially simpler programs relative to the program implementing the target rule for a given task. These results hint at a resource-rational account, where people possibly adopt increasingly *simpler* rules when the target rule is costlier to infer or verify.

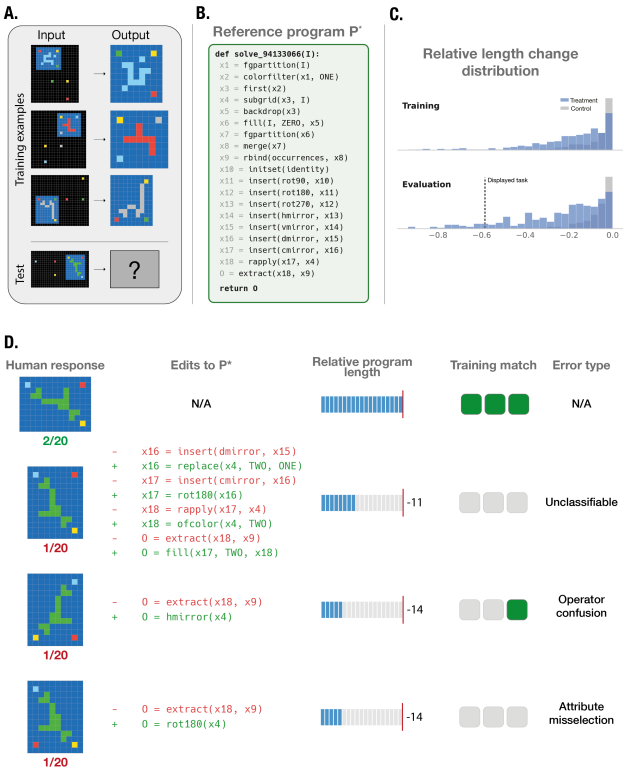


Fig. 2 Synthesized programs suggest people often infer rules simpler than the target rule for a given task. (**A**): A representative ARC task (`94133066`). Three training examples demonstrate the target rule—return the contents of the framed sub-grid under whichever of its eight rotations and reflections is consistent with the isolated colored cells scattered around it—followed by the test input for which participants construct an output grid (hidden with a grey square and "?" for illustrative purposes). (**B**): The reference program P^* implementing the target rule for this task. Because this task belongs to the `evaluation` split, P^* is itself model-synthesized, rather than human-written. (**C**): Distribution of relative program length change, $(L(p) - L(P^*))/L(P^*)$, over all synthesized programs, shown separately for the `training` and `evaluation` splits of ARC-AGI-1. Treatment programs (blue) are those conditioned on incorrect human responses; control programs (grey) are produced by an identical procedure conditioned instead on the correct output grid. Because edits replace lines in-place and causally irrelevant code is subsequently removed, synthesized programs can either shorten, or remain the same, but never lengthen. Therefore, the control condition controls for the shortening bias attributable to the procedure itself, against which the leftward displacement of the treatment distribution should be read. The dashed line marks the median shortening of modeled responses for the task displayed in this figure. (**D**): The correct response (top row) and the three of the most frequent incorrect responses to that task, with response frequencies below each grid (green, correct; red, incorrect). For each, we show the diff with P^* after applying the synthesis procedure, the length of the resulting program relative to P^* (blue, retained lines; gray, removed lines; red rule, reference length), which training examples the program reproduces (green, reproduced), and its error category. A single edit can account for a large simplification: for example, replacing the final `extract` over the eight candidate transforms by one fixed transform renders fourteen lines of the reference program inconsequential, and they are removed. Thus, the simpler resulting program hardcodes a transform instead of exhaustively searching over possible transformations.

1.3 Inferred programs partially satisfy available evidence

Because we represent the rules that people inferred as programs, we can verify how these programs behave on the training examples that participants observed. For each program that reproduces a human response and its associated task, we executed it on each of the training input grids to determine whether it produced any correct output grids. We find that overall, 22.7% (94% HDI [22.0%, 23.5%]) of modeled incorrect attempts correspond to programs consistent with at least one training example, without any explicit constraints on the synthesizer to produce such programs and minor differences between the two splits (see Supplementary Table S3 for per-split results). This subset of programs is split between two rule types: partial-match and alternative rule (Figure 3C, top). Partial-match programs (20.9%, 94% HDI [20.2%, 21.6%]) correspond to rules where a strict subset of training outputs are correctly generated by the program. Conversely, alternative rules, programs that correctly reproduce all observed training examples and the behavioral response but not the test output, captured 1.8% (94% HDI [1.5%, 2.1%]) of modeled attempts. The first incorrect response in Figure 3D illustrates a partial-match rule: the program that captures it selects the rightmost object rather than the least common one, a rule that happens to satisfy the first two training examples but fails on the third.

We further examined synthesized programs for order effects: do partial-match programs suggest order-dependent inference or verification of rules? We classified each partial-match program by which examples it reproduces: the first or the last k examples shown on screen, a contiguous block of at least two examples, and anything else as "Other" (Figure 3C, bottom). On the `training` split, 62.5% (94% HDI [59.4%, 65.6%]) of partial-match programs form a structured subset, dominated by the first-k pattern at 41.0% (94% HDI [38.2%, 44.0%]), followed by 17.0% (94% HDI [14.8%, 19.4%]) matching the last-k and 4.4% (94% HDI [2.9%, 5.9%]) a contiguous block; the remaining 37.5% (94% HDI [34.4%, 40.6%]) match a single interior example or a non-contiguous set. On the `evaluation` split, the structured share is lower at 48.3% (94% HDI [45.7%, 51.1%]), with 29.9% (94% HDI [27.6%, 32.3%]) first-k, 15.8% (94% HDI [14.1%, 17.8%]) last-k, and 2.4% (94% HDI [1.5%, 3.5%]) contiguous block.

Our results suggest that people often find rules that are incomplete explanations of the data they observe. Further, we hypothesize that such rules may be the consequence of ordered, sequential rule induction and/or verification, paired with early stopping. Taken together, these results are consistent with satisficing [43, 49]: people adopt a rule based on whether it satisfies a "good enough" threshold, rather than verifying it against all evidence or expending more cognitive effort to find better, potentially more complex rules that explain all the observed data.

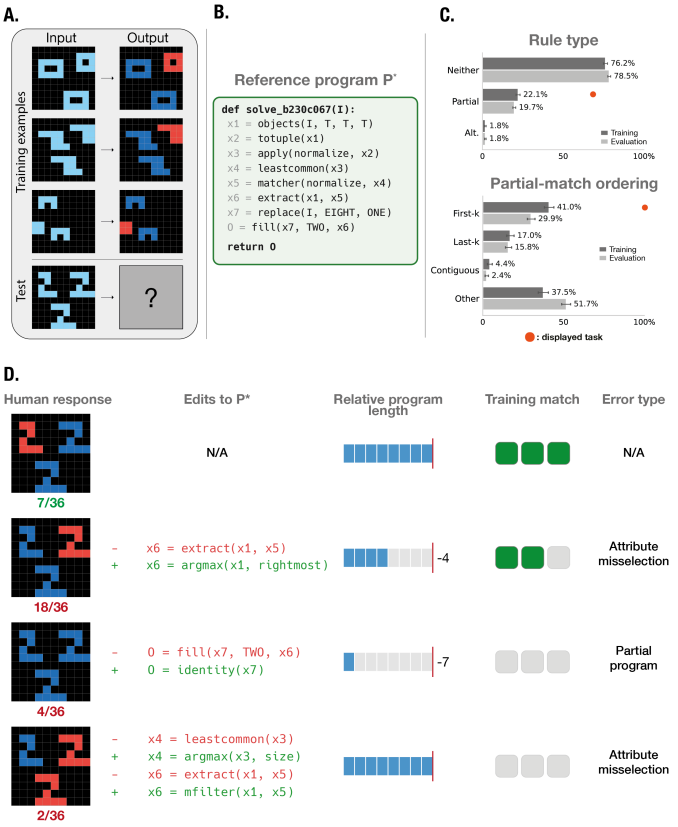


Fig. 3 Synthesized programs suggest people infer rules that only explain some of the data they observe. (**A**): A representative ARC task (`b230c067`). Three training examples specify the target rule—recolor every shape blue except the least common one, which is recolored red—followed by the test input for which participants construct an output grid (hidden with a grey square and "?" for illustrative purposes). (**B**): A human-written reference program P^* implementing the target rule for this task. (**C**): Top, prevalence of each rule type among all modeled incorrect attempts. A program is a *partial-match* if it reproduces at least one, but not all, of the training outputs when executed on training inputs; an *alternative rule* if it reproduces all of them yet still fails the test example; and *neither* otherwise. Bottom, conditional on an attempt being explained by a partial-match program, the positions of the training examples it reproduces: the first-k or last-k example(s) shown on screen, a contiguous block of at least two examples that doesn't include the first or last examples, or any other configuration. In both plots, bars give the posterior median attempt-weighted prevalence and error bars the 94% HDI, shown separately for the `training` (dark) and `evaluation` (light) splits of ARC-AGI-1; orange dots give the corresponding proportion for the task shown in panel A. (**D**): The correct response (top row) and three of the most frequent incorrect responses to that task, with response frequencies below each grid (green, correct; red, incorrect). For each, we show the diff with P^* after applying our synthesis procedure, the length of the resulting program relative to P^* (blue, retained lines; gray, removed lines; red rule, reference length), which training examples the program reproduces (green, reproduced), and its error category. The most frequent incorrect response is captured by a program that selects the rightmost object rather than the least common one, a rule that satisfies the first two training examples but not the third.

Table 1 Taxonomy of error categories used to classify synthesized programs. Each category identifies a way in which programs that reproduce incorrect responses deviate from the reference program implementing the target rule for the same task. Categories are defined over structural differences between the two programs, but the classifier is also conditioned on the response grids. Categories are defined as mutually exclusive and the "unclassifiable" catch-all makes the set exhaustive. The examples in the third column are schematic illustrations of the contrast between reference and response programs that defines each category. Every synthesized program was classified by an LLM given a reference program, task grids, the synthesized program, and its execution trace on the training and test inputs, with five independent classifications sampled per program (§3.4).

Category	**Definition**	**Example**
Attribute misselection	The correct operation is applied, but with a misselected attribute, parameter, or input value.	inferred color → hardcoded color
Scope restriction	The correct computation is applied, but only to a subset of the entities or regions where it should apply.	all objects → subset
Operator confusion	The correct class of operation is identified, but the wrong member within that class is selected.	rotate CW → rotate CCW
Transformation misselection	The correct targets are identified, but a transformation from an entirely different computational family is applied.	mirror → recolor
Sequencing	All required operations are identified, but they are composed in the wrong order or bound to the wrong targets.	(A then B) → (B then A)
Partial program	A subset of the required steps are executed correctly; one or more computational blocks are entirely missing.	extract + tile → extract only
Boundary precision	The correct quantity is computed from the correct source, but with a small additive offset (off-by-one).	$h \to h - 1$
Unclassifiable	The response reflects an idiosyncratic rule with no recognizable structural relationship to the reference program.	ad-hoc rule

1.4 A taxonomy of reasoning errors

Prior work has sought to characterize reasoning error kinds, for example on intelligence tests [30]. Contrary to success rate, which is a sparse signal indicating differences in *degree*, error analyses afford characterizing differences in *kind*, which is informative about underlying cognitive strategies. Unlike the data we model here, researchers often collect forced-choice responses (e.g., Raven's Progressive Matrices) rather than richer, free-form responses. This implies that the scope of possible error types is predefined by the structure of the incorrect alternative responses of each task. In comparison, here we can more richly characterize the space of reasoning errors that people make, without strong constraints on their responses.

To classify types of errors, we devised a novel error taxonomy comprised of seven cognitive categories and a catch-all "unclassifiable" category (see Table 1). For each valid synthesized program for tasks in the `training` split, we sampled labels using an LLM classifier. We then aggregated program-level classifications at the attempt level using a hierarchical Bayesian model (§3.4). Overall, our classification procedure found a dominant category of errors: *attribute misselection* at 45.2% (94% HDI [43.9,

46.6]) of modeled incorrect attempts (Figure 4C). Inspired by [44]'s work on attribute substitutions in intuitive judgment, we define attribute misselections as errors that arise when a reasoner fails to identify or compute the correct dependency between an attribute and a target parameter. The first incorrect response shown in Figure 3D demonstrates an example of such a rule: determining which object to color red (i.e., the target parameter) as a function of which object is odd with respect to the others (i.e., the attribute). Instead, participants often rely on possibly simpler or more salient dependencies. Observing that the top-right object is always red is a correct inference based on the first two examples, but it does not hold in the third example. This error category aims to capture simplifications of this kind, and our results suggest that people often rely on such heuristics.

Additionally, we find that people often infer rules that correspond to *partial programs*, representing 11.9% (94% HDI [10.7, 13.2]) of modeled incorrect attempts. Such errors arise when participants correctly applied a rule that captures a subset of the relevant steps for solving a task. Finally, we find that 10.8% (94% HDI [9.8, 11.7]) of incorrect attempts are transformation misselections, where the participant applied an incorrect transformation to the right parts of the input. The remaining four cognitive categories represent less than 5% of modeled incorrect attempts, while 22.3% (94% HDI [21.5, 23.3]) are determined to be unclassifiable under our taxonomy.

Every modeled response we display is labeled with its modal error category (panel D, last column of Figures 2–4), illustrating the different ways in which participants can deviate from a single target rule. For task `7fe24cdd` in Figure 4, the rule is to tile the input into a 2×2 arrangement of successive $90°$ rotations. In that figure's panel D, the first response omits the rotations and simply tiles the input four times (*partial program*, four lines shorter), the next preserves the construction but substitutes reflections for rotations (*transformation misselection*, same length), and the third applies the construction to a subset of the input (*scope restriction, one line shorter*). Notably, the three deviations differ in length—a relationship we examine next.

Beyond the growing relative shortening predicted as a function of target rule complexity (§1.2), our results suggests substantial variation in relative shortening across error categories (§3.8; see Supplementary Table S4). Around a grand-mean shortening of 9.2% (94% HDI $[7.6, 10.8]$), the seven cognitive categories span roughly 34 percentage points: at one pole, partial-program errors produce the greatest simplification with programs 32.3% shorter than the target rule ($\gamma = -0.231$, 94% HDI $[-0.264, -0.198]$), while at the other, boundary precision errors yield essentially no reliable simplification, with predicted length statistically indistinguishable from the reference ($\gamma = +0.111$, 94% HDI $[0.065, 0.159]$).

Overall, we find that our analysis recovers an interpretable hierarchy between the categories. This result suggests that when people make simple localized errors such as boundary precision or operator confusion, the rules they infer, as modeled by our programs, approximate much of the structure of the target rule. Conversely, when people only partially execute the rule or select the wrong transformation, their approximations are coarser, which manifests as substantially shorter programs.

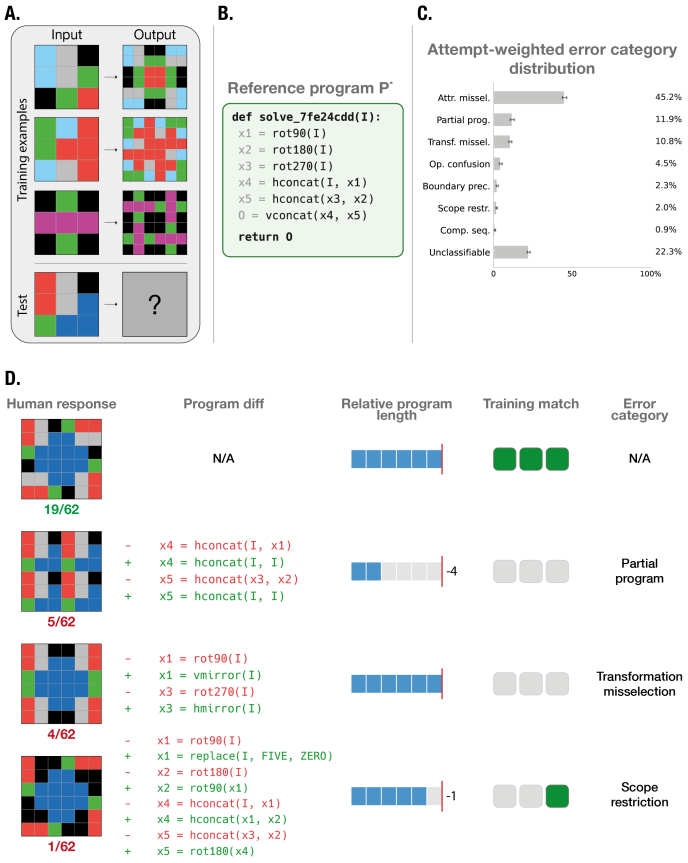


Fig. 4 Synthesized programs reveal a restricted set of recurring deviations from target rules. (**A**): A representative ARC task (`7fe24cdd`). Three training examples specify the target rule—tile the input into a 2×2 arrangement in which each quadrant holds the input rotated by a further 90° clockwise, starting from the input at the top left—followed by the test input for which participants construct an output grid (hidden with a gray square and "?" for illustrative purposes). (**B**): A human-written reference program P^* implementing the target rule for this task. (**C**): Prevalence of each error category among modeled incorrect attempts. An LLM classifier assigned every program one of seven cognitive categories, given the reference program, the task grids, the synthesized program and its execution trace (Table 1 for definitions). Bars give the posterior median of the attempt-weighted prevalence derived from a hierarchical Bayesian categorical model, and error bars the 94% HDI; the categories are mutually exclusive. Classification was performed on programs fitting responses to `training` split tasks only. (**D**): The correct response (top row) and three of the most frequent modeled incorrect responses to the task in panel A, with the response frequency below (green, correct; red, incorrect). For each, we show the diff with P^* after applying our synthesis procedure, the length of the resulting program relative to P^* (blue, retained lines; gray, removed lines; red rule, reference length), which training examples the program reproduces when executed on their inputs (green, reproduced), and its error category. The programs that reproduce these three incorrect responses diverge from P^* in different ways: omitting the rotations and tiling the input four times, carrying out reflections instead of rotations, or correctly carrying out the rule on a restricted portion of the input.

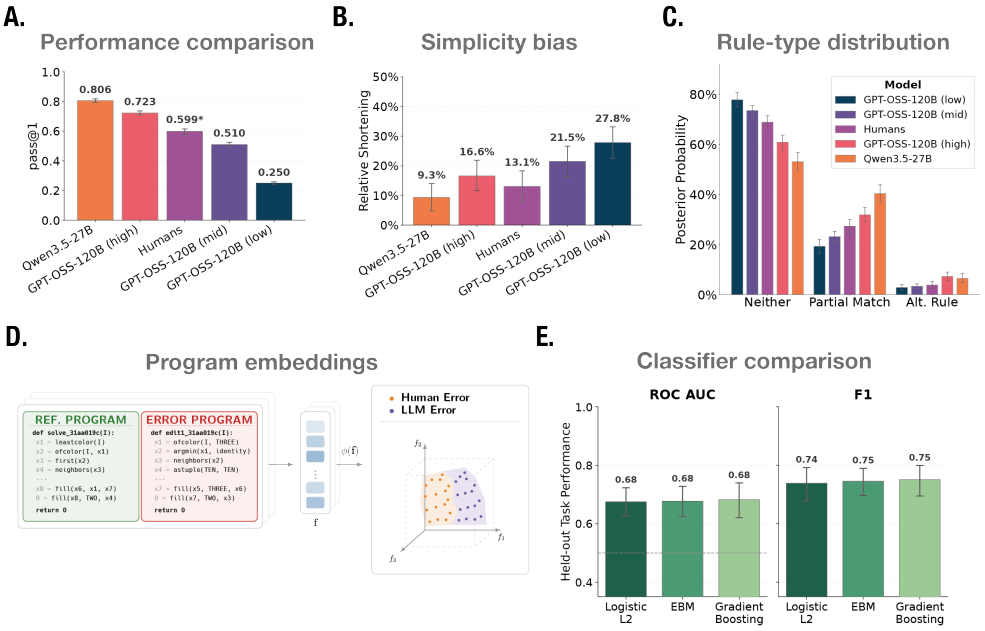


Fig. 5 Machine errors are systematic and performance-dependent. (**A**): Performance (pass@1) of LLMs at different reasoning efforts and humans on the `training` set of ARC-AGI-1. Note that GPT-OSS-120B uses a mixture-of-experts architecture and only has a total of 5.1 billion active parameters during inference. (**B**): Relative shortening of programs synthesized from model errors compared with programs for correct-output controls; all model configurations exhibit a simplicity bias, which is stronger for lower-performing configurations. (**C**): The proportion of error programs that partially match or fully reproduce the support examples increases with model performance. (**D**) Schematic of the classification analysis distinguishing human from LLM errors using features derived from synthesized error programs. (**E**) Above-chance performance of three fitted classifiers at distinguishing human from LLM errors on held-out tasks using linearized program-level feature vectors, measured by F1 score and ROC-AUC. Panels **B** and **C** report results on the subset of 60 tasks where both reasoning language models and people generated incorrect responses (see Appendix S5 for similar trends across all errors).

1.5 Comparing human and machine errors

We study two reasoning-tuned LLMs, Qwen3.5-27B [50] and GPT-OSS-120B [51], both of which generate long chains of thought (CoTs) before producing an answer. Because GPT-OSS-120B allows us to vary how much the model dynamically allocates tokens for CoT before providing an answer (i.e., test-time compute), we evaluate it at high, medium, and low reasoning effort. We sample ten grids for each model and configuration on each of the 400 `training` tasks in ARC-AGI-1 (§3.6).

Figure 5A shows the mean pass@1 rate and its standard deviation across runs. Performance varies substantially across models and reasoning efforts. Qwen3.5-27B, with approximately 27 billion active parameters, performs best at 80.63% pass@1, followed by GPT-OSS-120B, with approximately 5.1 billion active parameters, at 72.28% under high reasoning effort. Both models outperform the estimated average human accuracy

of 59.9% on a single attempt, as reported by [47]. Reducing reasoning effort substantially lowers GPT-OSS-120B's performance to 50.96% at medium effort and 25.02% at low effort. After filtering (§3.1), we observe 4,166 model-unique errors across all configurations, spanning 357 of the 400 tasks.

Our pipeline captures a large fraction of model errors, indicating that many LLM errors admit systematic programmatic descriptions. After verification and filtering (§3.1), we obtain 3,804 programs that capture 54.2% (991/1830) of GPT-OSS-120B unique errors at low reasoning effort, 2,760 programs that capture 52.0% (714/1374) at medium reasoning effort, and 944 programs that capture 46.7% (236/505) of its errors at high reasoning effort. For Qwen3.5-27B, 823 programs capture 43.1% (197/457) of its unique errors. For comparisons across models and humans, we restrict subsequent analyses to the 60 tasks on which humans and every model configuration make at least one error, thereby avoiding task difficulty as a confounding factor. Similar trends hold when considering the full pool of errors across models (see Supplementary Tables S6–S8).

We investigate whether LLM errors exhibit a similar simplicity bias as observed in people by looking at whether programs synthesized from reasoning-tuned LLM errors are shorter than those synthesized for the correct-output control group (§1.2). Figure 5B shows the average relative shortening of error programs compared with baseline programs. Two notable patterns emerge. First, all model configurations exhibit a simplicity bias similar to humans, but to a varying degree. For the strongest model we tested, Qwen3.5-27B, its error programs are 9.3% shorter on average than baseline programs ($\beta = -0.093$, 94% HDI $[-0.140, -0.048]$, $P(< 0) = 1.00$). For GPT-OSS-120B, error programs are 16.6% shorter at high reasoning effort ($\beta = -0.166$, 94% HDI $[-0.219, -0.116]$, $P(< 0) = 1.00$), 21.5% shorter at medium reasoning effort ($\beta = -0.215$, 94% HDI $[-0.266, -0.163]$, $P(< 0) = 1.00$), and 27.8% shorter at low effort ($\beta = -0.278$, 94% HDI $[-0.332, -0.226]$, $P(< 0) = 1.00$). Second, the magnitude of relative shortening is correlated with pass@1 performance: programs that represent errors of lower-performing systems are shorter relative to baseline than those of higher-performing systems (Pearson's $r = -0.92$, permutation $p = 0.042$, $n = 5$). These results provide model-based support for our hypothesis that simplification is driven by resource constraints in people: less capable systems, or those devoting less compute to reasoning, tend to infer simpler rules.

As with people, we also analyze the prevalence of rule types and order effects (§1.3) within programs synthesized from LLM errors. Figure 5C shows that the proportion of partial-match rules increases with model capacity, while alternative rules are substantially more frequent for the two highest-performing configurations. For GPT-OSS-120B at low reasoning effort, only 19.3% (94% HDI $[16.4\%, 22.0\%]$) of programs produce partial matches and 2.8% (94% HDI $[1.7\%, 3.9\%]$) represent alternative rules. For Qwen3.5-27B, these proportions rise to 40.3% (94% HDI $[37.0\%, 43.9\%]$) and 6.5% (94% HDI $[4.8\%, 8.4\%]$), respectively (Supplementary Table S7 for all values across the full set of errors). Only GPT-OSS-120B at high reasoning effort shows a human-like order effect, with more first-k than last-k partial matches (35.0%, 94% HDI $[30.0\%, 40.2\%]$ vs. 23.3% $[18.5\%, 28.4\%]$; cf. §1.3). The other configurations show no consistent preference for explaining the first examples, with GPT-OSS-120B

at medium reasoning effort instead leaning towards last-k matches (Supplementary Figure S4B and Supplementary Table S7). These findings suggest that the latest generation of more capable LLMs make more systematic errors than was previously found [52]. Similarly to people, models find simpler, often incomplete rules that nonetheless capture meaningful aspects of the underlying task structure.

Furthermore, we find that the distributions of error types show a similar relationship with performance. As in humans, *attribute misselection* (see §1.4 for the taxonomy) is the most frequent error type across models, and its prevalence increases with performance. It accounts for 63.82% (94% HDI [60.60%, 67.08%]) of Qwen3.5-27B errors and 60.60% (94% HDI [57.88%, 63.60%]) of GPT-OSS-120B errors at high reasoning effort, compared with 49.70% (94% HDI [47.26%, 51.98%]) and 42.31% (94% HDI [39.54%, 45.00%]) at medium and low effort, respectively. Lower-performing models instead produce a larger proportion of *unclassifiable* errors (Supplementary Figure S4A and Supplementary Table S8), suggesting less systematicity.

Finally, we investigate whether meaningful structural differences can differentiate human and LLM errors based on their synthesized programs. We trained classifiers to predict whether an error was made by a human or an LLM based on simple, linearized program-level features (see Figure 5D and §3.6). All classifiers achieve performance above chance on held-out test tasks (Figure 5E), with EBM and gradient boosting performing equally well (F1 = 0.75, ROC-AUC = 0.68, for both models). Thus, although human and machine errors share broad patterns in simplicity bias, rule-type distributions, and error categories, measurable differences remain in the specific primitive operations that are used to reproduce their respective errors. This suggests that possible differences in primitive-cost profiles and inductive biases support rule-search in people versus reasoning language models.

2 Discussion

Understanding how people reason in more open-ended and cognitively complex domains is a central goal of cognitive science [53–55]. Although symbolic models have desirable properties for advancing cognitive theories, recent advances in AI have brought into question the prospects of symbolic models of cognition due to their lack of scalability. Here, we address this gap by proposing a method which leverages these advances, and allows us to scale symbolic modeling to a cognitively challenging domain for both AI and people. Crucially, although our method does not provide an algorithmic or process-level account, it allows us to retrieve process-level signatures by characterizing inferred rules using interpretable and structured formal objects. In particular, we find that the programs we reverse-engineer provide bottom-up support for accounts of reasoning where people are thought to engage in satisficing [43]: instead of searching for the single best rule consistent with all the evidence, people often settle on rules that are less costly to represent, verify, or compute. We find that the programs that explain people's rules are systematically simpler than the reference programs implementing target rules, and that this simplification grows with the complexity of the reference program. Further, we find that a substantial proportion of modeled attempts correspond to programs consistent with at least one example, suggesting sequential

accumulation of evidence or verification of hypothesized rules. Finally, attribute misselections, where a reasoner incorrectly infers attribute dependencies, is the modal error category under our taxonomy.

Studying the behavior of reasoning-tuned LLMs on these problems allows us to evaluate to what extent models behave like people, and explore possible underlying mechanisms for the patterns we observe. We find that reasoning effort, and problem-solving capacity more generally, modulate our results in the expected directions. Specifically, we show that the amount of compute a model devotes to chain-of-thought reasoning strongly correlates with the simplicity of the programs that explain the model's inferred rules: models with lower reasoning effort infer increasingly simpler rules. On the contrary, models with more reasoning effort, increasingly find better rules that match at least some of the observed demonstrations and fall within the scope of our cognitive error categories, thus capturing more regularities of observed training examples. Paired with increased overall performance on ARC problems in recent years, the reasoning language models we evaluate here appear to be far more systematic than was previously observed or suggested [52]. Combined with our human results, these patterns suggest a common underlying rule-search mechanism that is systematically graded by allocation of resources, with differences in inductive biases between people and machines, supported by our human-machine program classifier analyses. This conclusion about resource-constrained reasoning is further supported by other research suggesting a correspondence between the number of reasoning tokens LLMs allocate to problems and thinking time in humans on the H-ARC dataset [56]. Together, our findings hint at a resource-rational characterization of reasoning in open-ended domains [57], though establishing this formally would require an explicit account of the costs being traded off in people and machines [48].

Our results hinge on a methodological choice: modeling incorrect responses. Correct answers without additional behavioral measures provide limited evidence about how people come about solutions, whereas incorrect ones can provide a window into underlying cognitive mechanisms: departures from normative probability have exposed the heuristics underlying intuitive judgment [44], characteristic failures in syllogistic reasoning make predictions that support adjudicating between competing cognitive theories of deductive inference [58], and children's overregularization of inflectional morphology has revealed the acquisition of productive rules rather than the memorization of forms [59]. Similarly, our results speak to the shape of the limitations that people face in more open-ended reasoning domains. Among synthesized programs that correctly reproduce some but not all of the observed data (i.e., "partial-rules"), the covered subset of examples often suggests sequential, ordered evaluation of hypotheses (i.e., "first-k" rule types). Noisy or randomly truncated evaluation of the examples would not produce this ordered asymmetry, and exact Bayesian inference is order-invariant. In contrast, sequential sampling algorithms as proposed by [60] predict a dependence on presentation order. Relatedly, because all training examples remain visible throughout solving, it is unlikely that the pattern is attributable to degraded encoding of examples. However, whether people form hypotheses from a limited subset of the examples, or form them more broadly but only verify them sequentially against part of the evidence remains an open question for future work.

Similar to prior research in cognitive science, our method assumes that people's inferences are best understood as deviations from a normative model [61–63]. In our work, the normative standard takes the form of programs written in a DSL. We operationalized deviations from this standard by adapting programs into error programs through LLM-generated line edit proposals, and restricting edits using a locality prior: resulting error programs should remain reasonably close in line edit space to reference programs. This strategy has early precedents, for example in the diagnosis of children's arithmetic mistakes, where errors were explained as deviations from a correct procedure by manually defining libraries of "bugs" (e.g., procedural misconceptions) [28, 29]. More broadly, other work analyzing individual deviations from normative models typically attribute rules to individuals using a pre-specified candidate set: distractors written by a test designer [27, 30, 64], hand-built libraries of misconceptions [28, 29], or hypothesis spaces small enough to enumerate over binary features [5]. Using an LLM backbone as a proposal distribution and conditioning program synthesis on the observed response bypasses this requirement, and allows individual-level rule attribution in a domain where neither the space of possible responses nor the space of possible rules can be enumerated. However, our approach also carries a caveat specific to programs as normative objects. The reference programs we utilize are one of many equivalent ways of expressing the target rule in the DSL we use. As such, quantities defined against it—edit distance, relative program length—are defined relative to a particular encoding rather than to the rule itself. Two considerations mitigate this concern: reference programs for the `training` split were written independently of the present analysis [46], and the effects we report also hold on the `evaluation` split, whose reference programs were obtained under an entirely different procedure (§3.2.1).

Typically, program induction approaches to cognitive modeling hand-engineer the hypothesis space of cognitive models using primitives which are believed to capture mental operations relevant for solving a given experimental task [10, 23–26].Similarly, we leverage a hand-engineered DSL [46] to express our programs. An important limitation of such approaches is that the primitives are posited a priori instead of being discovered or inferred [22]. While many of the operations in our DSL are cognitively plausible (e.g., geometric transformations, object groupings, and other spatial predicates), others have no clear cognitive counterpart (e.g., type coercion). Recent advances in automated library learning [24, 65, 66] provide a promising direction for automatically generating structured hypothesis spaces, potentially guided by human-generated natural language descriptions [38, 67, 68]. We leave this for future work. Additionally, our method suffers from a lack of response-level attribution scoring: multiple distinct synthesized programs often reproduce the exact same output grid. Multiple dimensions such as program complexity and cognitive cost [48] possibly modulate which program best captures the underlying mental model an agent constructed while solving a given problem. We leave for future work how one might weight programs to adjudicate between computationally equivalent candidate rules. Finally, while our framework reverse-engineers symbolic artifacts that capture what rule an agent may have executed, it does not model process-level dynamics by which an agent arrives at a rule—online search algorithms, working memory bounds, or sequential hypothesis-testing steps. Addressing process-level considerations will require integrating dynamic

behavioral metrics such as response time, eye-tracking as well as carrying out controlled experimental conditions to further illuminate how human reasoners search and revise candidate rules; for language models, reasoning traces potentially offer a complementary and analogous window.

Our work builds on multiple strands of research from AI and cognitive science. Recent advances have led to impressive developments in science automation [69–73], with particular applications of this line of work to automated cognitive model discovery [35, 36, 39, 40, 74, 75]. Here, we develop similar tools to scale longstanding methods of psychological inquiry which have often faced a scalability bottleneck [76]. Inspired by seminal work such as Newell & Simon's [55] early attempts to develop theories of human problem-solving by analyzing detailed, individual-level behavioral protocols, we instantiate a similar but automated approach using AI tools. More broadly, this work demonstrates the feasibility and prospects of scaling symbolic modeling to complex, more open-ended domains. By leveraging neural models as flexible and efficient proposal distributions over discrete program spaces, we can automate symbolic modeling to systematically uncover representational biases and process-level signatures that can inform cognitive theories. Further, our approach demonstrates that distinctive cognitive footprints can be decoded from the structure of synthesized programs that explain behavior in different systems. We see exciting prospects for the application of our approach to other domains for understanding how different black-box systems solve complex problems, shedding light on both human and machine cognition.

3 Methods

3.1 Data and filtering

We use the H-ARC dataset to understand how people reason on abstract rule-induction problems [45, 47]. The dataset consists of output grid responses across 800 distinct ARC-AGI-1 few-shot visual rule inference problems from over 1700 online participants. For the scope of the human results reported here, we focus our modeling efforts on a total of 723 ARC-AGI-1 tasks (362 tasks from the `training` split; 361 from the `evaluation` split), yielding a total of 6261 unique incorrect output grids across 1387 participants and 7402 attempts.

We pre-processed the H-ARC dataset by identifying all near-misses and "trivial" errors, corresponding to 15.6% (1,222/7,809) of the full set of incorrect unique grids, and filtered them out under a noise tolerance criterion. Qualitative analyses revealed that for larger grids, "errors" could often be plausibly attributed to procedural mistakes (i.e., mis-clicks; Supplementary Figure S1). Given our modeling approach and goals, capturing this kind of noise within programs would not be informative about rule inference since we aim to understand conceptual mistakes. As such, we apply a noise tolerance criterion such that for any "incorrect" response grid of ten or more cells, we filter it out as a "near-miss" if it matches the correct test output grid with up to two mismatched cells; of the 353 unique near-miss grids we found, 51.3% differed from the correct output by a single cell and 47.9% by exactly two (Supplementary Figure S2, Supplementary Table S1). Additionally, participants sometimes copied the test input grid (7.9% of unique responses) or training inputs and outputs (3.5% of

unique responses). For test-input copies we applied the same tolerance criterion—for instance, if a participant produced a 5×2 output identical to the test input grid modulo one or two incorrect cells, we filtered it out as an identity error, whereas copies of a training example were removed only on an exact match. For grids smaller than ten cells, we apply a strict filter by only removing grids that are exact copies of the test input or training examples. Finally, we remove all "empty" 3×3 grids (1.9% of unique responses) because this grid state corresponds to the initial state of the test output grid in our task interface (i.e., a blank response).

3.2 Domain-specific language and reference programs

We model rules as programs in a domain-specific language (DSL) for ARC. To do so, we leverage an expressive open-source DSL which was released alongside 400 human-written reference programs implementing the correct rule for each task [46]. The DSL consists of 160 primitive functions spanning object detection and manipulation (e.g., `objects`, `ofcolor`, `recolor`, etc), grid transformations (e.g., `rot90`, `fill`, etc), spatial queries (e.g., `height`, `adjacent`, `manhattan`, etc), selection and ranking (e.g., `argmax`, `mostcommon`, etc), and higher-order compositional and programmatic operations (e.g., `compose`, `fork`, `branch`, etc).

The DSL is typed over a small set of object-centric types built from Python immutable types: grids as nested tuples of integers, objects as nested frozen sets of colored cells, etc. Additionally, the DSL contains a small set of primitive constants: a ten color palette, boolean values, and directional constants.

Programs in this DSL are written in A-normal form (ANF; [77]), a canonical form in which every non-trivial sub-expression is named via direct assignment, and function arguments must be atoms (i.e., variables or constants). The programs also respect static single assignment, where every variable is named once and only once. Although there are many possible expressions of correct programs for a given task rule, we focus on using the 400 human-written ones available from the open-source DSL for the `training` split. For the remaining 400 tasks from the `evaluation` split, we use the shortest synthesized solver for tasks where we successfully synthesized at least one solver.

3.2.1 Synthesizing additional reference programs for full coverage

As described above, the DSL we utilize in this work was released with 400 human-written solvers for half of the publicly available tasks from ARC-AGI-1, excluding the 400 tasks in the `evaluation` split. To obtain full coverage of H-ARC incorrect attempts, we prompted an LLM to generate synthetic solvers for the remaining 400 tasks. We successfully generated solvers for 361 tasks of the 392 tasks from the `evaluation` split of ARC-AGI-1 that have incorrect responses. To do so, we simply prompted `Gemini Pro 3.1` (Supplementary Figure S6) using retrieval-augmented generation [78] with five in-context examples of other tasks and corresponding human-written solvers from the `training` split. For each queried `evaluation` task, we sampled five independent responses, and selected the shortest program from the set of valid and correct DSL programs for each task.

3.3 Symbolic model synthesis

Behavior-conditioned symbolic model synthesis. Our approach involves automating inference over behavioral outputs to produce programs representing the rules that an agent may have used to generate its responses. To so do, we leverage an LLM by prompting it to synthesize a program representation of each incorrect attempt. We sample from the LLM structured outputs of line edits restricted to a JSON schema, and its responses on task grids, a complete DSL reference, the reference program for the in-context task, and an incorrect output grid produced by a human participant or an LLM (Supplementary Figure S5). The LLM is instructed to generate a restricted set of edits to the in-context reference program such that the resulting program would produce the observed response when executed on the test input grid. The edits consist of in-place edits to referenced lines of code from the reference program. The LLM is instructed to generate syntactically valid, ANF lines of code written exclusively using the DSL. For each unique incorrect response, we collect 5 independent samples with edit budgets of 2, 4 and 8 lines, yielding a total of 15 samples per error. For human responses, we obtained a total of 93,855 program samples (40,365 for the `training` set split responses; 53,490 for the `evaluation` set split responses) across 723 tasks. Samples were generated using `Gemini 3 Pro` for the `training` split and `Gemini 3.5 Flash` for the `evaluation` split, with temperature set to 1.0, reasoning budget set to high, and default parameters otherwise.

For each sampled structured output of edits, we programmatically replace each tagged line with its corresponding edited line, and verify that the resulting programs respect ANF syntactic constraints and exclusively use DSL operators. Any edits that violate the instructions are rejected as invalid and discarded. Often, editing one or multiple lines has the side-effect of disconnecting variables in the reference program from the computation graph processing the input into the output grid. Thus, each program is rewritten using a custom parser which removes these pieces of code after applying the edits. We then execute each program using the test input grid for the corresponding task, and verify its output against the target error grid. We only accept a program as an explanation for a given error if it produces an output that is equivalent to the closest match from the pool of generated programs under the same noise tolerance criterion as for filtering errors (§3.1 for more details). Concretely, we apply a noise tolerance criterion such that for any synthesized program, if its output grid has ten or more cells, we consider it a valid model of the observed response if it matches the target response grid with up to two mismatched cells. For example, for a given target output grid, if the best program in the pool of 15 samples produces a correctly sized output that is at least 10 cells large, but differs by a single cell from the target, then any other program from that pool must produce outputs that are equally accurate to be accepted. After filtering, we obtained a total of 8,657 unique valid programs explaining human errors (3,690 for the `training` set split responses; 4,967 for the `evaluation` set split responses; see §3.6 for machine errors). Across modeled errors, we computed the per-error average minimum edits: the minimum number of edits averaged across errors. This serves as an upper bound on the shortest path in line-edit space from the reference program for explaining a given response.

Exhaustive depth-1 program enumeration. To characterize the space of programs adjacent to human-written ARC solutions, we exhaustively enumerated all single-edit mutations of the 400 `training` split reference programs. Because the programs are written in ANF and respect static single assignment, the set of syntactically well-formed edits is finite and enumerable. We adapted code from previous work [79] where a stochastic search algorithm over DSL program mutations in the ARC setting was proposed as a form of data augmentation to train deep learning models for program induction.

Each program is parsed into an abstract syntax tree and a concrete type is inferred for every variable by propagating the types of the task's first training input through the DSL's primitive signatures. For every assignment statement in the program, we then enumerated four kinds of type-correct edits:

- Argument replacement: substituting one argument of the call for any other in-scope variable or primitive constant of the same type
- Function replacement: substituting the called primitive for any primitive with a compatible output type
- Right-hand-side replacement: substituting the entire expression for any type-correct application of a DSL primitive to in-scope variables and constants
- Argument replacement with variable introduction: inserting a new assignment immediately before a given statement and passing its result as the argument, with downstream variables renumbered accordingly

This procedure yielded a total of 16,661,032 candidate programs. We then executed each candidate on all input grids of its parent task. To do so, we wrote a custom line-by-line interpreter with a 0.25s per-statement timeout and a 1000-cell bound on grid and intermediate-state size, discarding executions that raised, diverged, or exceeded the size bound. Only candidates producing a valid output grid on both the training and test inputs were retained. This left 9,422,295 executable programs (56.6% of those enumerated). Enumeration and execution were parallelized across 52 CPU cores and completed in approximately 16 hours on the NYU high-performance computing cluster. Programs were determined to match a human error under the same noise tolerance criterion as LLM-generated programs.

3.4 Error classification

To characterize the kinds of inferential shortcuts that people and machines utilize to solve ARC tasks, we developed a taxonomy of error types based on comparisons between programs that explain human and machine responses, and the reference programs from which they were adapted (see Table 1). Although some errors only require single-line edits, or even simpler node-level edits such as swapping an argument or a primitive function for another, the majority of synthesized programs that reproduce human responses required multi-line edits. Such program differences are difficult to programmatically interpret since their consequences can span different parts of the computation graph, and as a result they cannot be easily attributed to a particular high-level cognitive category without semantic interpretation or the contextual information from the training examples. Thus, we developed a precise taxonomy of seven

error categories which we believed would capture meaningful cognitive constructs, and prompted an LLM (Supplementary Figure S7) to categorize each program from the final filtered collection of programs that match human or machine responses.

For each program, we prompt an LLM to return a structured JSON schema output containing one of eight possible labels: seven cognitive categories and an "unclassifiable" catch-all category. In addition to definitions for each category, we include the following elements in the model's prompt: the reference program for the given task, the task training input and output grids, the synthesized program, and the synthesized program execution trace on the training and test input grids. This final portion explicitates how the program behaves on the training examples, as well as what test output grid it produces (i.e., a human or machine response). Concretely, the classifier must assign a label to structural differences between the reference program and the error program while taking into account the task data. For instance, a sub-tree that computes height relative to color count might instead be collapsed to a constant integer value in the error program, which coincides with some training examples. We sampled 5 independent classifications per program, yielding 18,450 classifications for the 3,690 programs that explain human errors on the `training` split, and 41,655 classifications for the 8,331 programs that explain machine errors (4,115 for Qwen3.5-27B; 4,720, 13,800, and 19,020 for GPT-OSS-120B at high, medium, and low reasoning effort, respectively). All samples were generated using `Gemini 3.1 Pro`, with temperature set to 1.0, reasoning budget set to high, and default parameters otherwise.

3.5 Pipeline validation

To assess the validity and sources of bias of our pipeline, we manually constructed a validation set of synthetic errors by mechanically producing targeted, simple edits on the full set of human-written and synthetic reference programs. For each of the eight categories, we defined a function that implements a limited set of mutation operators, applies them to a given input program, and returns new, modified programs that fall under the syntactic constraints of the corresponding error category. In total, this yielded 2,375 synthetic errors across the 8 categories. We then manually verified a small random subset of errors for each category (444 in total) to obtain a target of $\sim$20 errors per category, balanced between the human-written solvers of the `training` split and the LLM-synthesized solvers of the `evaluation` split. Manual verification is required to ensure semantic interpretation of errors conditioned on task grids. In total, we obtained 148 synthetic errors which we use for validation, with attribute misselection containing 19 programs, and sequencing only 9. We refer to the errors generated and verified using the procedure just described as validation errors/programs. Validation errors refer to the grids obtained by executing the validation programs.

We passed the validation errors through the full pipeline to evaluate how accurately the predicted categories matched the gold-labeled ones. Overall, the pipeline achieved end-to-end category agreement of 64.9%, 75.0%, and 76.4% at edit budgets of $k \in \{1, 2, 3\}$, respectively. We evaluated these budgets because validation program edit distances are known by construction—124 of the 148 errors are single-line edits, 141 are reachable within two edits, and all 148 within three—though in practice the true edit distance is unknown. To understand what governs this performance and why

accuracy plateaus around 76%, we decompose the pipeline into three potential sources of loss: classification error (*labeling*), search failure (*recovery*), and semantic shift under alternative programs (*aliasing*).

We first isolated labeling by running the classifier directly on the validation programs, which establishes an accuracy ceiling of 85.1%. Classifications across five runs per program were highly consistent at $\sim$95% agreement, and we used modal labels as the unit of analysis. Second, we evaluated whether search failure accounts for the drop from this ceiling. At adequate budgets, it does not. Increasing k from 1 to 3 raised recovery from 84.5% to 99.3% (95.3% at $k = 2$), successfully finding a candidate program for all but one validation error grid. Instead, the remaining gap is driven almost entirely by program aliasing. Across all budgets, the pipeline recovered the **exact** injected program in roughly two-thirds of cases (62%–65%). When search found an alternative, aliased program instead, the classifier still identified the injected category in 57.3% of cases (remaining essentially flat at 52.6%–59.4% across budgets). Thus, program aliasing rather than search failure or classifier degradation, accounts for the primary drop between the 85.1% ceiling and end-to-end performance.

3.6 Evaluating machine errors

As described in §1.5, we evaluate two reasoning-tuned LLMs on the `training` split of ARC-AGI-1 to study their error behavior: Qwen3.5-27B [50] in thinking mode and GPT-OSS-120B [51] at high, medium, and low reasoning effort. We evaluate each model configuration across ten random seeds using a standard evaluation prompt (see Supplementary Figure S8), on all 416 test inputs of the 400 tasks (4,160 responses per configuration). H-ARC participants attempted only the first test input of the 14 tasks that have two; none of the 60 matched tasks used in §1.5 has more than one test input. For Qwen3.5-27B, we follow the recommended sampling settings for general tasks in thinking mode with a temperature of 1.0, a top-p value of 0.95, a top-k value of 20, a minimum-p value of 0.0, a presence penalty of 1.5, and a repetition penalty of 1.0. For GPT-OSS-120B, we follow GPT-OSS's official vLLM example and use a temperature of 1.0, a top-p value of 1.0, disabled top-k filtering, a minimum-p value of 0.0, a presence penalty of 0.0, and a repetition penalty of 1.0 across all three reasoning-effort settings. We set a maximum output budget of 65,536 generated tokens, including reasoning and the final answer, for both models. For GPT-OSS-120B, we control reasoning behavior through the reasoning-effort setting without imposing a separate reasoning-token budget. For Qwen3.5-27B, we additionally impose a reasoning-token budget of 49,152 tokens to limit time and resources spent on inference. If Qwen3.5-27B reaches this budget before ending its reasoning, the end-of-thinking marker `</think>` is appended to its current output, terminating the reasoning block and allowing final-answer generation within the remaining output budget. We observe that only 4.35% (181/4160) of responses reach the model's maximum reasoning budget.

A response is counted as correct only if generation finishes within the maximum token budget, and the extracted grid exactly matches the target in both dimensions and cell values. We parse predicted grids by selecting the last valid group of bracketed grid rows, accepting spaces or commas between entries, and falling back to parsing Python/JSON-style nested lists. A valid grid must be nonempty, rectangular, and

contain integer values from 0 to 9. Completed responses from which no valid grid can be extracted are classified as parsing errors. Both unfinished responses and completed responses with no valid grid remain included in the overall accuracy calculations but are excluded from subsequent analyses (see Supplementary Table S6). We find that Qwen3.5-27B generates 806 incorrect responses, of which 689 contain valid but incorrect grid predictions. For GPT-OSS-120B, 742/1153 incorrect responses at high reasoning effort contain valid but incorrect grid predictions, compared with 2002/2040 at medium reasoning effort and 3059/3119 at low reasoning effort. After removing incorrectly formatted grid outputs, we applied the additional filtering steps described in §3.1 (see Supplementary Table S5 for the full per-configuration breakdown).

We pass each incorrect grid prediction to the behavior-conditioned symbolic model synthesis pipeline described in §3.3. As with human responses, we collect 5 independent program samples at edit budgets of 2, 4, and 8 lines, yielding a total of 15 samples per machine error. In total, we obtain $8,625$ program samples for errors from Qwen3.5-27B and $9,780$ samples for GPT-OSS-120B at high reasoning effort, $26,070$ at medium reasoning effort, and $37,365$ at low reasoning effort. Samples are generated using `Gemini 3.5 Flash`, with the temperature set to 1.0, the reasoning budget set to high, and all other parameters left at their default values. All programs are subsequently verified and filtered as described in §3.3. After filtering, we obtain 823 unique valid programs for Qwen3.5-27B's errors, 944 programs describing GPT-OSS-120B's errors at high reasoning effort, $2,760$ at medium reasoning effort, and $3,804$ at low reasoning effort. For comparisons across models and humans, we restrict our analyses in §1.5 to the 60 tasks on which humans and every model configuration make at least one error, thereby avoiding task difficulty as a confounding factor. On these 60 tasks, Qwen3.5-27B produces 117 errors captured by 499 programs, while GPT-OSS-120B produces 137 errors described by 627 programs at high reasoning effort, 232 errors described by $1,037$ programs at medium reasoning effort, and 215 errors described by 975 programs at low reasoning effort. All statistical models in §3.8 are fit separately for each model configuration and for humans, using only the errors and programs from these 60 tasks unless stated otherwise.

3.7 Predicting human versus machine errors

We train classifiers on program features to predict whether an error was made by a human or an LLM. As described in §3.3, DSL programs are written in A-normal form as sequences of primitive function calls. We extract two feature sets describing differences between these programs (Figure 5D).

Primitive delta vectors. $\mathbf{v}_\Delta \in \mathbb{Z}^{160}$ record the signed difference in call counts between the error-producing and reference programs, with one entry per DSL primitive. For example, if one `rotate180` call is replaced by `hmirror` and all other calls remain unchanged, the vector contains -1 for `rotate180`, $+1$ for `hmirror`, and zeros elsewhere.

Primitive category transition vectors. $\mathbf{v}_\rightarrow \in \mathbb{N}_0^{84}$ describe how individual edits change the type of operation performed. We group primitives into eight semantic categories: *functional operations* (function composition, application, and control flow),

perception (object detection), *selection* (selecting and filtering), *measurement* (computing attributes), *geometry/spatial operations* (coordinates and transformations), *grid/rendering operations* (constructing and modifying grids), *representation* (converting object representations), and *scalar/container operations* (arithmetic, logic, and collections). For each recorded edit that changes an expression's root operation, we increment the count for its directed transition from the original category to the replacement category. Non-primitive endpoints, such as constants and variable references, receive separate labels. The vector contains 82 observed directed-transition counts and two aggregate counts for within-category and cross-category changes. For example, replacing `mapply` with `ofcolor` increments both the *functional* $\rightarrow$ *geometry/spatial* entry and the cross-category count. Unchanged calls contribute no transition, and changes induced by subsequent cleanup are excluded.

We fit L2-regularized logistic regression, explainable boosting machines (EBMs), and histogram gradient boosting classifiers on the extracted features. Each candidate program explaining an observed error provides one training example. Source-balanced weights allocate half the total training weight to humans and half to LLMs, with the LLM share divided equally among the four LLM configurations. Within each source, the weight is divided equally among tasks, then among errors within each task, and finally among candidate programs within each error. Logistic regression features are scaled by training-fold standard deviations without centering; boosting models use raw values.

We use nested cross-validation with five outer and five inner folds, keeping all errors and candidate programs from a task together at both levels to avoid task information leakage. We select hyperparameters to maximize mean per-task validation ROC-AUC, calculated after averaging candidate probabilities within each error. We search $C \in \{0.01, 0.1, 1, 10, 100\}$ for logistic regression; $\{0, 5\}$ interactions and $\{100, 300\}$ boosting rounds for EBM; and $\{7, 15\}$ maximum leaf nodes and $\{100, 300\}$ iterations for histogram gradient boosting. Pre-processing and interaction selection use only data from the corresponding training fold.

Classifiers are trained on errors from all tasks, which maximizes the number of examples available to each fold. Reported metrics, however, are computed only on the out-of-fold predictions for the 60 tasks on which humans and every model configuration make at least one error (§3.6), so that the human and machine error sets being discriminated are drawn from the same tasks. For evaluation, we average candidate-level probabilities to obtain one out-of-fold prediction per observed error. We report pooled ROC-AUC and binary F1, treating LLM errors as the positive class and using a probability threshold of 0.5 for F1, together with the corresponding metrics averaged equally across tasks. We calculate 95% percentile confidence intervals from 10,000 bootstrap samples of whole tasks while keeping the out-of-fold predictions fixed. All classifier and feature comparisons use identical task folds and bootstrap resamples.

We compare classifiers trained on $\mathbf{v}_{\Delta}$, $\mathbf{v}_{\rightarrow}$, and their concatenation into a 244-dimensional vector, using the same candidate programs, errors, tasks, folds, and bootstrap resamples (Supplementary Table S9). For all three classifiers, the pooled ROC-AUC point estimate is higher with primitive delta features than with category transition features (0.65–0.68 versus 0.59–0.61). Concatenating the two feature sets

raises the pooled ROC-AUC point estimate for logistic regression (0.652 → 0.675) and EBM (0.652 → 0.677), but not for histogram gradient boosting (0.683 versus 0.682).

3.8 Statistical analyses

All analyses were performed using a Bayesian framework. Continuous outcomes were modeled as Gaussian linear mixed-effects regressions, using Bambi v0.17.2 for the main simplicity model and PyMC v5.28.5 directly for error category simplicity model; categorical outcomes were modeled as non-centered hierarchical categorical regressions directly in PyMC v5.28.5 with a softmax inverse link. All posteriors were approximated using the No-U-Turn Sampler (NUTS; [80]) via the JAX/NumPyro backend [81], with a target acceptance rate set to 0.9. We ran inference using 4 chains, with warm-up samples specified for each model below. For all models, we use non-centered parameterizations of the random effects, which is standard practice for inference using NUTS. Convergence was assessed by the rank-normalized split-$\hat{R}$ statistic and bulk/-tail effective sample size, using thresholds of $\hat{R} < 1.01$ and ESS > 400 per parameter; we additionally verified that no divergent transitions occurred. For every parameter of interest we report the posterior mean (or median for probabilities) and the 94% highest-density interval (HDI). For regression coefficients, we additionally report the posterior probability of direction $P(\theta > 0 \mid \mathcal{D})$, following the convention of [82].

3.8.1 Simplicity bias

To determine whether error-producing programs were simpler than baseline programs synthesized for correct outputs under an identical synthesis procedure, we fit a hierarchical Bayesian linear model. Let y_{ijk} denote the relative line-count change of the k-th synthesized program for unique error j in task i, and let $x_{ijk} \in \{0, 1\}$ indicate the treatment condition (where $x_{ijk} = 0$ for control and $x_{ijk} = 1$ for treatment). We compute the relative line-count change as $(L(p) - L(p^*))/L(p^*)$, where $L(p)$ is the line count of the synthesized program, and $L(p^*)$ is the line count of the reference program from which it was adapted. We modeled the outcome as:

$$\begin{aligned} y_{ijk} \mid \mu_{ijk}, \sigma &\sim \mathcal{N}(\mu_{ijk}, \sigma^2), \\ \mu_{ijk} &= \beta_0 + \beta_1 x_{ijk} + u_i + v_{ij}, \\ u_i &\sim \mathcal{N}(0, \tau^2_{\text{task}}), \\ v_{ij} &\sim \mathcal{N}(0, \tau^2_{\text{err}}) \end{aligned}$$

We include task-level random intercepts u_i to account for baseline variations in program length across ARC tasks, and nested per-error intercepts v_{ij} to handle the fact that each unique error contributes multiple candidate programs that are not independent observations. Thus, the model answers whether synthesized programs that fit incorrect human or machine responses, tend to be more or less complex than reference programs subjected to an identical procedure modulo the conditioned response data. We dummy-coded the condition factor with the control baseline as the reference level, so that the slope β_1 is directly interpretable as the treatment-minus-control contrast. Analyses were restricted to tasks present in both conditions to ensure the random-effects structure was identified.

For the comparison with machines (Figure 5B), all five fits share a single control set: the 140 programs synthesized for the correct test outputs of the 60 matched tasks (§3.6).

Finally, we placed weakly informative priors on all coefficients and variance components:

$$\begin{aligned} \beta_0 &\sim \mathcal{N}(0, 1.5^2), \\ \beta_1 &\sim \mathcal{N}(0, 1.5^2), \\ \tau_{\text{task}},\, \tau_{\text{err}} &\sim \text{Half-Normal}(1) \end{aligned}$$

The residual standard deviation σ was assigned `Bambi`'s default data-scaled prior.

3.8.2 Rule types and order effects

Given error-producing programs, we modeled how often such programs also produce the correct output grids for at least one training example but not all (i.e., partial-match programs), or all the training examples but not the test example (i.e., alternative rules). To do so, we fit a hierarchical categorical Bayesian model, and each program was run on all the training examples of its associated task and labeled according to the outputs it produced. Let $y_{i,j} \in \{1, \ldots, K\}$ be the rule-type classification of program j for unique error i, where $K = 3$ categories: "alternative rule" (set as reference, $k = 1$), "neither" ($k = 2$) and "partial-match" ($k = 3$). We modeled the likelihood and linear predictors as:

$$\begin{aligned} y_{i,j} \mid \boldsymbol{p}_i &\sim \text{Categorical}(\boldsymbol{p}_i), \\ p_{i,k} &= \text{softmax}(\boldsymbol{\eta}_i)_k, \\ \eta_{i,1} &= 0 \quad \text{(reference)}, \\ \eta_{i,k} &= \beta_{0,k} + u_{i,k} \quad k = 2, \ldots, K, \\ u_{i,k} &\sim \mathcal{N}(0, \tau_k^2) \end{aligned}$$

where $\boldsymbol{p}_i = (p_{i,1}, \ldots, p_{i,K})$ is the error-specific probability vector on the $(K-1)$-simplex, shared by all programs j belonging to error i. As in the previous model, we include per-error intercepts $u_{i,k}$ since there are often multiple programs per error, each with a possibly different behavior on the training examples. The "alternative rule" category is fixed at zero as the reference, since the softmax link is translation invariant. Finally, we placed weakly informative priors on the coefficient and variance components:

$$\begin{aligned} \beta_{0,k} &\sim \mathcal{N}(0,\ 2.5^2), \\ \tau_k &\sim \text{Half-Normal}(1) \end{aligned}$$

With this model, we seek to understand how likely it is that an incorrect rule is modeled by a program that also satisfies some or all of the constraints imposed by the training examples human or machine reasoners observed. Accordingly, we define the attempt-weighted rule-type prevalence over the empirical error pool as a derived quantity of the model parameters:

$$P_k(\boldsymbol{\theta}) = \sum_{i=1}^{N} w_i \cdot \mathrm{softmax}([0,\ \eta_{i,2},\ \eta_{i,3}])_k\,, \qquad w_i = \frac{c_i}{\sum_{i'} c_{i'}}$$

where $\boldsymbol{\theta} = (\boldsymbol{\beta}_0, \boldsymbol{\tau}, \{\boldsymbol{u}_i\}_{i=1}^{N})$, $\eta_{i,k}$ is defined as above, c_i is the number of attempts that produced error i (the number of participant attempts for human data; for model configurations, the number of sampled responses), and N is the total number of unique errors. Evaluating P_k at each posterior draw $\boldsymbol{\theta}^{(s)}$, $s = 1, \ldots, S$, yields a Monte Carlo sample $\{P_k(\boldsymbol{\theta}^{(s)})\}_{s=1}^{S}$ from the posterior distribution of the prevalence, which we summarize by its median and 94% HDI. Because each error contributes its realized random effect $\boldsymbol{u}_i$ weighted by w_i, P_k corresponds to the expected rule-type prevalence one would observe when sampling incorrect attempts uniformly at random from the human or machine incorrect response distributions.

To characterize which output structure is reproduced by the partial-match rules, we fit a second instance of the same hierarchical categorical model, restricted to the partial-match programs, on ordering categories of the reproduced training pairs. For a partial-match program that reproduces $0 < k < n$ of the n training outputs, we assign an ordering category based on the positions of the matched pairs: "first-k" when they form a block anchored at the first training example (similar to a primacy pattern), "last-k" when the block is anchored at the last example (similar to a recency pattern), "contiguous" when the run is interior (length ≥ 2, not including the first or last example), and "other" for a single interior match or a scattered, non-contiguous set. Let $o_{i,j} \in \{1, \ldots, K'\}$ be the ordering category of partial-match program j for unique error i, where $K' = 4$ and "contiguous" is the reference ($k = 1$). The likelihood, softmax link, per-error random effects $u_{i,k}$, and priors are exactly as in the rule-type model above; only the observation set (partial-match programs), the category set, and hence the number of categories change. We summarize the same attempt-weighted prevalence $P_k(\boldsymbol{\theta})$ by its posterior median and 94% HDI, with the weights $w_i = c_i / \sum_{i'} c_{i'}$ renormalized over the errors that admit at least one partial-match program. The resulting prevalences are therefore conditional on an error being explained by a partial-match program, and describe, among such attempts, how the reproduced training examples are distributed by position.

3.8.3 Error categories

Similarly to the rule-type model described above, we modeled the prevalence of the different error types among the programs that reproduce incorrect human or machine attempts on ARC tasks. For each program, we first obtain five classification samples using our LLM classifier (§3.4). Given this pool of samples, each unique response may have between 5 and $5M_{\max}$ label samples, where $M_{\max}$ is the largest number of unique error programs retained for a single error. Let $y_{ijs} \in \{1, \ldots, K\}$ be the error category assigned by sample s to program j of unique error i, where $K = 8$ categories: "attribute misselection" (set as reference, $k = 1$), "boundary precision" ($k = 2$), "sequencing" ($k = 3$), "operator confusion" ($k = 4$), "partial program" ($k = 5$), "scope restriction" ($k = 6$), "transformation misselection" ($k = 7$), and "unclassifiable" ($k = 8$). We modeled,

$$
\begin{aligned}
y_{i,j,s} \mid \boldsymbol{p}_{ij} &\sim \text{Categorical}(\boldsymbol{p}_{ij}),\\
p_{ij,k} &= \text{softmax}(\boldsymbol{\eta}_{ij})_k,\\
\eta_{ij,1} &= 0 \quad \text{(reference)},\\
\eta_{ij,k} &= \beta_{0,k} + u_{i,k} + v_{ij,k} \quad k = 2, \ldots, K,\\
u_{i,k} &\sim \mathcal{N}(0, \tau_k^2),\\
v_{ij,k} &\sim \mathcal{N}(0, \omega_k^2)
\end{aligned}
$$

The difference with the rule-type model is the additional program-level random effect $v_{ij,k}$, nested within the per-error effect $u_{i,k}$, and the number of categories. We include both effects because each unique error may be explained by several distinct programs, and each program is in turn labeled multiple times by the classifier: the per-error intercepts $u_{i,k}$ absorb systematic differences between errors, while the program-level intercepts $v_{ij,k}$ absorb variation between the programs offered for the same error together with the labeling noise across the replicated samples. As before, "attribute misselection" is fixed at zero as the reference, since the softmax link is translation invariant. We placed the same weakly informative priors on the coefficient and both variance components:

$$
\begin{aligned}
\beta_{0,k} &\sim \mathcal{N}(0,\ 2.5^2),\\
\tau_k, \omega_k &\sim \text{Half-Normal}(1)
\end{aligned}
$$

As with the rule-type model, we define the attempt-weighted error-category prevalence over the empirical error pool as a derived quantity of the model parameters:

$$
P_k(\boldsymbol{\theta}) = \sum_{i=1}^{N} w_i \cdot \text{softmax}([0,\ \tilde{\eta}_{i,2}, \ldots, \tilde{\eta}_{i,K}])_k\,, \qquad w_i = \frac{c_i}{\sum_{i'} c_{i'}}
$$

where $\boldsymbol{\theta} = (\boldsymbol{\beta}_0, \boldsymbol{\tau}, \boldsymbol{\omega}, \{\boldsymbol{u}_i\}_{i=1}^N, \{\boldsymbol{v}_{ij}\})$, $\tilde{\eta}_{i,k} = \beta_{0,k} + u_{i,k}$ is the error-level linear predictor evaluated with the program-level effects held at zero ($v_{ij,k} = 0$), c_i is the number of attempts that produced error i (the number of participant attempts for human data; for model configurations, the number of sampled responses), and N is the total number of unique errors. Evaluating P_k at each posterior draw $\boldsymbol{\theta}^{(s)}$, $s = 1, \ldots, S$, yields a Monte Carlo sample $\{P_k(\boldsymbol{\theta}^{(s)})\}_{s=1}^S$ from the posterior distribution of the prevalence, which we summarize by its median and 94% HDI. Holding $v_{ij,k} = 0$ evaluates each error at its typical program; the reported prevalences therefore describe the error category one would expect for a typical program explaining an incorrect attempt sampled uniformly at random from the human or machine incorrect response distributions.

3.8.4 Relative length change and error categories

Having established that error programs are simpler than the reference programs they are adapted from (§3.8.1), we next asked how this relative shortening varies with reference program length, task difficulty, and error type. We fit a Bayesian linear regression predicting the relative line-count change of each error program, with reference program line count and empirical task success rate as continuous covariates and the error

category as an additional term. Since we only ran error classification on programs fitting incorrect responses to tasks from the `training` split, this model is only run on that subset of programs. Let y_{ijr} denote the relative line-count change (defined as in §3.8.1) of the r-th error program for unique error j in task i, let s_i be the H-ARC empirical success rate of task i, and let $\ell_i = L(p_i^*)$ be the line count of the task's reference program. Both covariates were mean-centered, with $\bar{s}$ and $\bar{\ell}$ denoting their sample means. We modeled the outcome as:

$$\begin{aligned} y_{ijr} \mid \mu_{ijr}, \sigma &\sim \mathcal{N}(\mu_{ijr}, \sigma^2), \\ \mu_{ijr} &= \beta_0 + \beta_1(s_i - \bar{s}) + \beta_2(\ell_i - \bar{\ell}) + \boldsymbol{\theta}_{ijr}^\top \boldsymbol{\gamma} + u_{ij}, \\ u_{ij} &\sim \mathcal{N}(0, \tau_{\text{err}}^2) \end{aligned}$$

As in the previous models, we include per-error random intercepts u_{ij} to account for the several candidate programs that may explain a single unique error. Centering the covariates makes β_0 the expected relative length change at the mean success rate and mean reference length. The slopes β_1 and β_2 capture how relative shortening covaries with task difficulty and reference program complexity, respectively.

The vector $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_K)$ is a per-category effect on relative length change for the $K = 8$ categories of §3.8.3, and $\boldsymbol{\theta}_{ijr}$ is a fixed K-vector of category weights for program r. Instead of collapsing the classifier's five samples per program to a single majority-vote label, we carry that uncertainty into the covariate. Let $\boldsymbol{c}_{ijr}$ be the program's category count vector across its n_{ijr} classification samples. We set $\boldsymbol{\theta}_{ijr}$ to the Dirichlet–multinomial posterior mean under a symmetric Dirichlet(α) prior,

$$\theta_{ijr,m} = \frac{c_{ijr,m} + \alpha}{n_{ijr} + K\alpha}, \qquad \alpha = 0.1,$$

so that the category effect enters the mean linearly as the probability-weighted average $\boldsymbol{\theta}_{ijr}^\top \boldsymbol{\gamma}$. We used a minimal smoothing constant ($\alpha = 0.1$) so that a program with unanimous classifications loads its weight almost entirely on its own category, keeping $\boldsymbol{\gamma}$ on the true per-category magnitude scale, while non-unanimous programs distribute weight across their sampled categories. Treating $\boldsymbol{\theta}_{ijr}$ as fixed data (rather than as a latent allocation informed by the length outcome) keeps the category term a plain linear-Gaussian covariate with unimodal geometry. Unlike the corresponding latent-mixture formulation which failed to converge, this model samples cleanly under NUTS.

The seven cognitive categories share a sum-to-zero prior, so that each of their effects is interpretable as a deviation from their common grand mean β_0. The "unclassifiable" category is held out of this constraint and given its own free effect. We set the model this way since this category collects short, degenerate programs whose shortening is several times larger than that of any cognitive category, and pooling it into the sum-to-zero prior would stretch that scale and distort the grand-mean anchor. We placed weakly informative priors on the coefficients and variance components:

$$
\begin{aligned}
\beta_0, \beta_1, \beta_2 &\sim \mathcal{N}(0,\ 1.5^2), \\
(\gamma_1, \ldots, \gamma_7) &\sim \text{ZeroSumNormal}(1.5), \quad \textstyle\sum_{m=1}^{7} \gamma_m = 0, \\
\gamma_8 &\sim \mathcal{N}(0,\ 1.5^2), \\
\tau_{\text{err}},\ \sigma &\sim \text{Half-Normal}(1.5)
\end{aligned}
$$

Because of the custom sum-to-zero structure on the category effects, this model was written and sampled directly in `PyMC` rather than through `Bambi`. We report the population-level coefficients β_1 and β_2, the per-category effects γ_m as deviations from the grand-mean relative length change β_0, and the absolute per-category predictions $\beta_0 + \gamma_m$. Predicted relative length changes at a given reference length are computed per posterior draw as $\beta_0 + \beta_2(\ell - \bar{\ell})$ and summarized by their mean and 94% HDI.

Supplementary information. This article includes Supplementary Information: Supplementary Notes S1–S7, Supplementary Figures S1–S6 and Supplementary Tables S1–S13.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

Declarations

Some journals require declarations to be submitted in a standardised format. Please check the Instructions for Authors of the journal to which you are submitting to see if you need to complete this section. If yes, your manuscript must contain the following sections under the heading 'Declarations':

- Funding
- Conflict of interest/Competing interests (check journal-specific guidelines for which heading to use)
- Ethics approval and consent to participate
- Consent for publication
- Data availability
- Materials availability
- Code availability
- Author contribution

If any of the sections are not relevant to your manuscript, please include the heading and write 'Not applicable' for that section.

Editorial Policies for:

Springer journals and proceedings: https://www.springer.com/gp/editorial-policies

Nature Portfolio journals: https://www.nature.com/nature-research/editorial-policies

Scientific Reports: https://www.nature.com/srep/journal-policies/editorial-policies

BMC journals: https://www.biomedcentral.com/getpublished/editorial-policies

Supplementary Information

This supplement reports (i) data for understanding the noise-tolerance criterion (§S1), (ii) per-split versions of every quantity reported as pooled results in the main text (§S2–§S4), (iii) per-model values for the human–machine comparison (§S5), and (iv) the prompts used at each stage of the pipeline (§S6).

Appendix S1 Filtering policy

Our filtering policy (§3.1) treats an incorrect response of ten or more cells as a near-miss, and filters it out as approximately correct, when it differs from the correct test output in at most two cells. We apply the same criterion for synthesized programs that approximately reproduce an error (§3.3). We provide some rationale for this two-cell constant in the pipeline.

Supplementary Figure S1 shows representative responses within the tolerance interval. In each case the participant's grid and the best-matching program output differ in one or two isolated cells against an otherwise identical structure.

Supplementary Figure S2 quantifies the same point over the full dataset. Panel A gives the coverage of incorrect attempts as a function of the tolerance τ: coverage is the empirical CDF of d, the minimum number of mismatched cells between any of the fifteen sampled programs for an error and the target response, restricted to size-matched outputs. At $\tau = 2$, coverage is 52.5% on the `training` split and 46.2% on the `evaluation` split. The curve keeps climbing beyond the cutoff, so the threshold is conservative. Panel B shows that the share of mismatched cells that are scattered singletons (no mismatched neighbor) falls from $\sim 60\%$ at $d = 2$ to $\sim 35\%$ by $d = 4$ and to $\sim 20\%$ beyond $d = 10$. Residuals of three or more cells are therefore likely to represent objects or other structures, but more analyses are needed to confirm this. In any case, accepting them would include programs implementing a different transformation, as opposed to a slightly imprecise one. Panel C shows the same cutoff from the filtering side: the distribution of cell-level distances between incorrect responses and the correct test output, with the removed near-miss data (≤ 2 cells) shaded.

Supplementary Table S1 reports what the trivial-error filter removes.

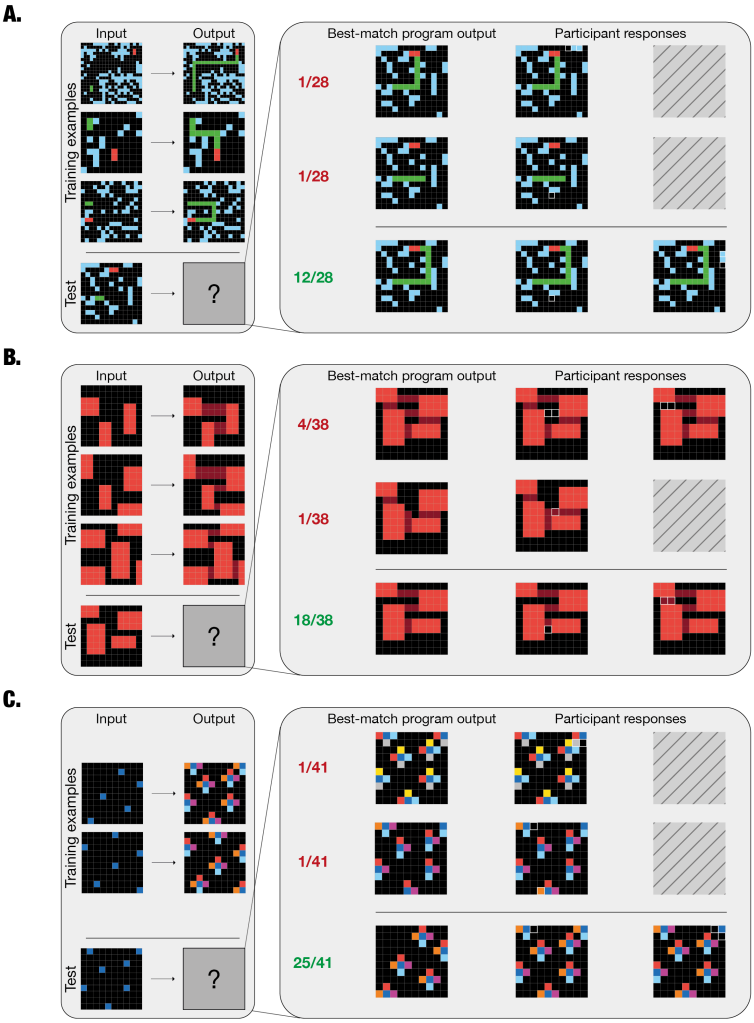


Fig. S1 Responses within the two-cell noise tolerance. (**A**–**C**) Three representative ARC tasks. Left: the training examples and the test input shown to participants (the test output is hidden with a grey square and "?"). Right: for three response grids per task, the output of the best-matching synthesized program (left column) beside the participant responses it is accepted as reproducing (remaining columns); hatched cells indicate that fewer than two distinct responses share that program. The fraction to the left of each row gives the number of participants producing that response out of all participants who attempted the task (red, incorrect; green, correct). Within each row the grids differ in at most two isolated cells (surrounded by white) against the best-match program output.

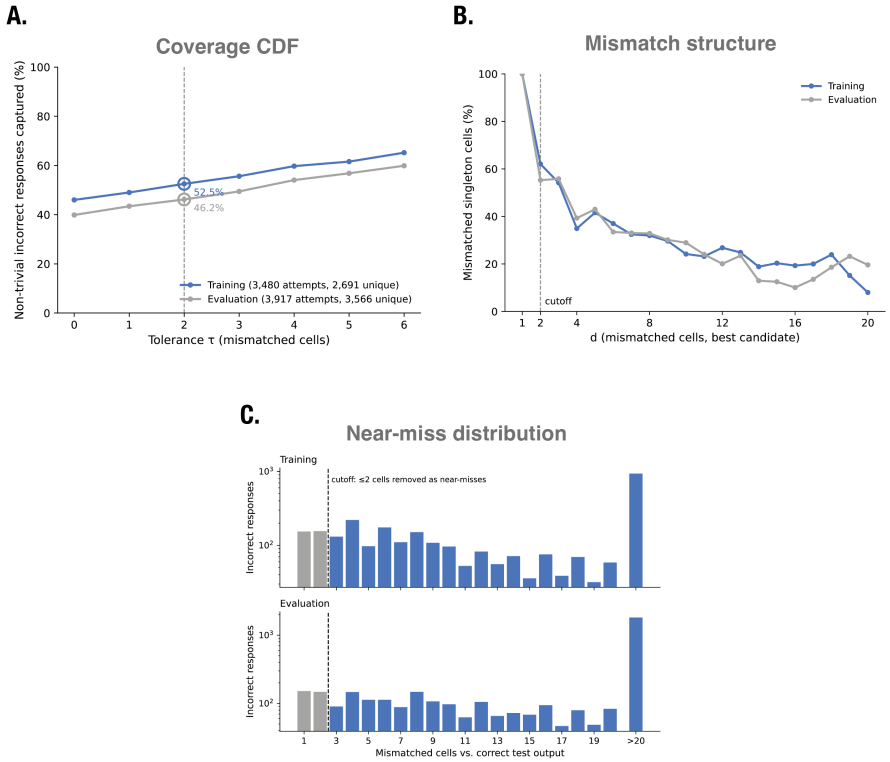


Fig. S2 The two-cell cutoff separates procedural noise from rule-level divergence. (**A**) Coverage of non-trivial incorrect responses as a function of the tolerance τ, attempt-weighted, shown separately for the `training` (blue) and `evaluation` (grey) splits. Circles mark the value $\tau = 2$ used throughout (52.5% and 46.2% respectively). (**B**) Share of mismatched cells that are scattered singletons—no mismatched cell among their four neighbours—as a function of d, the mismatch count of the best-scoring candidate program. The steep fall immediately past the cutoff shows that small residuals are isolated, misclick-like cells whereas may represent structured regions/objects. (**C**) Distribution of cell-level distances between incorrect responses and the correct test output, per split. Grey bars (≤ 2 mismatched cells) are the near-misses removed by the filter.

Table S1 Trivial-error filtering by split (§3.1). Counts and percentages of incorrect responses removed before modeling, under the noise tolerance criterion: i) shapes must match, ii) grids of ten or more cells tolerate up to two mismatched cells, iii) smaller grids must match exactly and iv) copies of a training example are removed only on an exact match at any size. Percentages are over the full pre-filter pool of responses of each split.

	Training	**Evaluation**	**Combined**
By incorrect responses (attempts)			
Incorrect responses	4,293	5,156	9,449
Filtered (all trivial)	808 (18.8%)	891 (17.3%)	1,699 (18.0%)
Near-miss (≈ correct)	189 (4.4%)	251 (4.9%)	440 (4.7%)
Test-input copy (identity)	368 (8.6%)	377 (7.3%)	745 (7.9%)
Training-example copy	130 (3.0%)	197 (3.8%)	327 (3.5%)
Empty 3×3 grid	121 (2.8%)	66 (1.3%)	187 (2.0%)
By unique incorrect grids			
Unique incorrect grids	3,220	4,589	7,809
Filtered (all trivial)	525 (16.3%)	697 (15.2%)	1,222 (15.6%)
Near-miss (≈ correct)	140 (4.3%)	213 (4.6%)	353 (4.5%)
Test-input copy (identity)	209 (6.5%)	282 (6.1%)	491 (6.3%)
Training-example copy	88 (2.7%)	142 (3.1%)	230 (2.9%)
Empty 3×3 grid	88 (2.7%)	60 (1.3%)	148 (1.9%)

Appendix S2 Modeling coverage and program yield by split

Supplementary Table S2 gives the per-split decomposition of the coverage results reported pooled in §1.1. `training` reference programs are hand-written whereas `evaluation` reference programs are model-synthesized (§3.2.1). Edits were sampled with `Gemini 3 Pro` on `training` and `Gemini 3.5 Flash` on `evaluation`.

Table S2 Modeling coverage and program yield by split. Unless specified, all reported numbers refer to errors modeled under the two-cell tolerance criterion. Attempt-weighted quantities weight each unique error by the number of participants who produced it. Exact-match reproduced is the percentage of errors with at least one program reproducing the grid with zero mismatched cells. Effective edits are counted after the parser removes causally irrelevant lines.

	Training	**Evaluation**	**Combined**
Reference program	hand-written	synthesized	—
Proposal model	Gemini 3 Pro	Gemini 3.5 Flash	—
Tasks modeled	362	361	723
Unique non-trivial errors	2,695	3,566	6,261
Incorrect attempts	3,485	3,917	7,402
Program samples collected	40,365	53,490	93,855
Tasks with ≥ 1 error modeled	304	304	608
Errors reproduced	1,274 (47.3%)	1,507 (42.3%)	2,781 (44.4%)
Attempts reproduced	1,827 (52.4%)	1,809 (46.2%)	3,636 (49.1%)
Participants modeled	548	611	1,159
Exact-match coverage (errors)	39.9%	35.5%	37.4%
Exact-match coverage (attempts)	45.9%	39.9%	42.7%
Total filtered programs	3,690	4,967	8,657
Programs per error (M $\pm$ SD)	2.90 $\pm$ 2.04	3.30 $\pm$ 2.57	3.11 $\pm$ 2.35
Programs per error (median [IQR])	2 [1, 4]	3 [1, 4]	2 [1, 4]
Programs per error (max)	10	20	20
Effective edits / program (M, SD)	3.30, 2.00	3.61, 2.14	3.48, 2.09
Effective edits, per-error min. (M $\pm$ SD)	2.69 $\pm$ 1.97	2.96 $\pm$ 2.09	2.84 $\pm$ 2.04

Appendix S3 Rule types and order effects by split

Supplementary Table S3 reports the per-split values behind the pooled rule-type and order-effect estimates of §1.3. Estimates are attempt-weighted population marginals from the hierarchical categorical model (§3.8.2). The prevalence of programs consistent with at least one training example is close across splits (23.8% versus 21.5%), whereas the structured share of partial matches is substantially higher on `training` (62.5% versus 48.3%), with the largest proportion coming from the first-k tier.

Table S3 Rule types and order effects by split (attempt-weighted). Posterior median and 94% HDI. A program is a *partial match* if it reproduces at least one but not all training outputs, an *alternative rule* if it reproduces all of them yet still fails the test example, and *neither* otherwise. Order effects are conditional on a partial match and describe which of the training examples the program reproduces, ordered as models or participants saw them.

	Training	**Evaluation**	**Combined**
n (attempts / errors)	1,827 / 1,274	1,809 / 1,507	3,636 / 2,781
Rule type (all modeled attempts)			
Neither	76.2% [74.9, 77.4]	78.5% [77.6, 79.4]	77.3% [76.5, 78.0]
Partial match	22.1% [20.8, 23.3]	19.7% [18.8, 20.6]	20.9% [20.2, 21.6]
Alternative rule	1.8% [1.3, 2.3]	1.8% [1.4, 2.3]	1.8% [1.5, 2.1]
Consistent with ≥ 1 example	**23.8% [22.6, 25.1]**	**21.5% [20.6, 22.4]**	**22.7% [22.0, 23.5]**
Order effects (conditional on a partial match)			
n (attempts / errors)	507 / 302	472 / 311	979 / 613
First-k (prefix run)	41.0% [38.2, 44.0]	29.9% [27.6, 32.3]	35.7% [33.9, 37.5]
Last-k (suffix run)	17.0% [14.8, 19.4]	15.8% [14.1, 17.8]	16.4% [15.1, 17.9]
Contiguous (interior run > 1)	4.4% [2.9, 5.9]	2.4% [1.5, 3.5]	3.4% [2.5, 4.4]
Other (single interior or scattered)	37.5% [34.4, 40.6]	51.7% [48.9, 54.3]	44.4% [42.4, 46.4]
Structured subset	**62.5% [59.4, 65.6]**	**48.3% [45.7, 51.1]**	**55.6% [53.6, 57.6]**

Appendix S4 Program length, reference complexity and error categories

Supplementary Figure S3 and Supplementary Table S4 report the full output of the program length model of §3.8.4, which regresses relative length change on reference-program line count and task success rate with a soft error-category term.

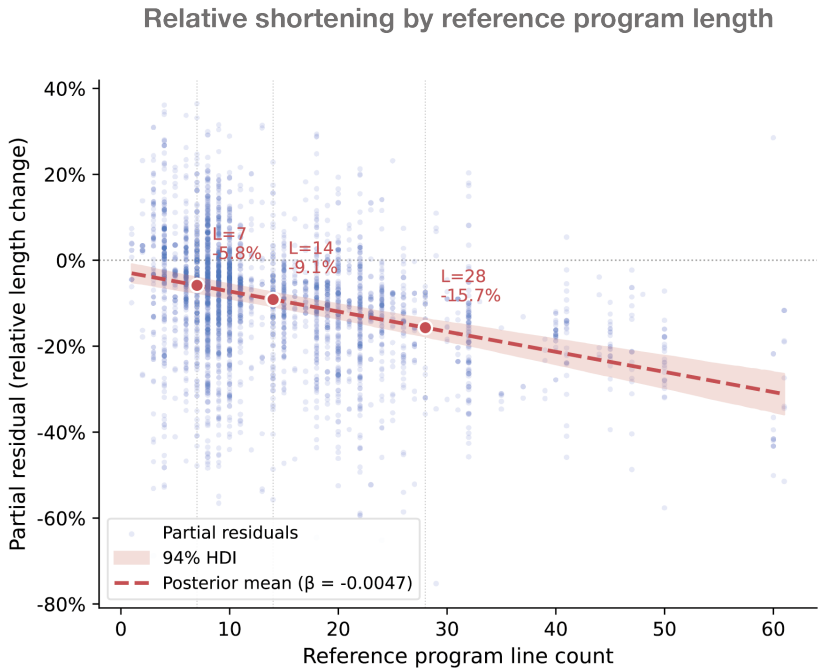


Fig. S3 Relative length change as a function of reference complexity and error category. Partial residuals of relative length change against reference-program line count, with the posterior mean slope (dashed) and its 94% HDI (band). Annotations mark the predicted shortening at $L = 7$, 14 and 28 lines.

Table S4 Error-category effects on relative program length (`training` split). γ_m is the deviation from the grand mean of the seven cognitive categories, which share a sum-to-zero prior; negative γ means more relative shortening than the average error type. The *unclassifiable* catch-all (†) is held outside that constraint. "% shorter" is the absolute per-category prediction $\beta_0 + \gamma_m$ at mean success rate and mean reference length, summed per posterior draw (grand-mean shortening $\beta_0 = 9.2\%$); a negative value means the error programs are *longer* than the reference. $n = 3{,}690$ programs over 1,274 errors.

Category	γ **[94% HDI]**	$P(> 0)$	**% shorter [94% HDI]**
Unclassifiable †	−0.369 [−0.395, −0.344]	0.000	+46.1 [+44.2, +48.0]
Partial program	−0.231 [−0.264, −0.198]	0.000	+32.3 [+29.1, +35.6]
Transformation misselection	−0.121 [−0.150, −0.091]	0.000	+21.3 [+18.4, +24.0]
Attribute misselection	+0.014 [−0.007, +0.034]	0.893	+7.8 [+6.2, +9.3]
Scope restriction	+0.045 [−0.006, +0.098]	0.947	+4.6 [−1.1, +10.4]
Operator confusion	+0.080 [+0.041, +0.118]	1.000	+1.2 [−2.8, +5.1]
Sequencing	+0.102 [+0.039, +0.164]	0.999	−1.1 [−7.9, +6.0]
Boundary precision	+0.111 [+0.065, +0.159]	1.000	−2.0 [−7.3, +3.0]

Appendix S5 Machine errors

Supplementary Table S5 reports what the trivial-error filter removes from each model configuration's incorrect responses before modeling, as the counterpart to Supplementary Table S1 for human responses. Supplementary Tables S6–S8 then report the extended LLM evaluation and associated error analyses on the full set of errors for comparison with the restricted-subset results reported in the main text. Figure S4 provides additional visualization of error analyses on the subset of tasks where both models and people produced modeled incorrect responses. Supplementary Table S9 reports pooled and task-averaged ROC-AUC and F1 for the human-versus-LLM source classifiers on this same subset, for each classifier and feature set (§3.7).

Table S5 Trivial-error filtering by model configuration (§3.1). Incorrect model responses with a valid grid removed before modeling, under the same noise tolerance criterion applied to human responses (Supplementary Table S1), counted by response (top) and by unique incorrect grid (bottom). Percentages are over each configuration's pre-filter total in the same block.

	Qwen3.5-27B	**GPT-OSS-120B**		
		high	**mid**	**low**
By incorrect responses (attempts)				
Responses with a valid grid	689	742	2,002	3,059
Filtered (all trivial)	114 (16.5%)	90 (12.1%)	264 (13.2%)	568 (18.6%)
Near-miss (≈ correct)	73 (10.6%)	54 (7.3%)	84 (4.2%)	61 (2.0%)
Test-input copy (identity)	22 (3.2%)	12 (1.6%)	110 (5.5%)	354 (11.6%)
Training-example copy	19 (2.8%)	24 (3.2%)	70 (3.5%)	153 (5.0%)
By unique incorrect grids				
Unique incorrect grids	536	562	1,522	2,059
Filtered (all trivial)	79 (14.7%)	57 (10.1%)	148 (9.7%)	229 (11.1%)
Near-miss (≈ correct)	57 (10.6%)	42 (7.5%)	72 (4.7%)	47 (2.3%)
Test-input copy (identity)	13 (2.4%)	7 (1.2%)	48 (3.2%)	110 (5.3%)
Training-example copy	9 (1.7%)	8 (1.4%)	28 (1.8%)	72 (3.5%)

Table S6 Evaluation and modeling yield by model configuration (across all tasks with errors). pass@1 is the mean over ten seeds on the 400 `training` tasks of ARC-AGI-1. Unique errors deduplicate identical grid outputs. Errors modeled is the count of errors with at least one verified program reproducing the error within the two-cell noise tolerance criterion—with coverage as a percentage of unique errors.

	Qwen3.5-27B	**GPT-OSS-120B**		
		high	**mid**	**low**
pass@1 (%)	80.63	72.28	50.96	25.02
Tasks with ≥ 1 error	145	173	284	344
Incorrect responses	806	1,153	2,040	3,119
with a valid grid	689	742	2,002	3,059
Unique errors	457	505	1,374	1,830
Program samples	8,625	9,780	26,070	37,365
Errors modelled	197 (43.1%)	236 (46.7%)	714 (52.0%)	991 (54.2%)
Filtered programs	823	944	2,760	3,804

Table S7 Simplicity bias, rule types and order effects by model configuration (across all tasks with errors). β is the posterior mean relative shortening of error programs against correct-output controls, where negative values indicate shorter programs. Rule types and order effects are attempt-weighted population marginals from the hierarchical categorical model (§3.8.2); posterior median and 94% HDI, in percent. A program is a *partial match* if it reproduces at least one but not all training outputs, an *alternative rule* if it reproduces all of them yet still fails the test example, and *neither* otherwise. Order effects are conditional on a partial match and describe which training examples the program reproduces, ordered as models or participants saw them.

	Qwen3.5-27B	**GPT-OSS-120B**		
		high	**mid**	**low**
Simplicity bias (relative length change vs. correct-output control)				
β	-0.129	-0.142	-0.195	-0.237
94% HDI	[−0.167,−0.090]	[−0.176,−0.107]	[−0.220,−0.169]	[−0.260,−0.214]
$P(\beta < 0)$	1.00	1.00	1.00	1.00
Rule type (all modeled attempts)				
Neither	59.1 [56.5,61.8]	60.2 [57.8,62.8]	78.6 [77.2,79.8]	84.2 [83.2,85.4]
Partial match	33.6 [31.0,36.2]	33.0 [30.5,35.4]	19.6 [18.3,20.8]	14.7 [13.6,15.7]
Alternative rule	7.3 [5.6,8.9]	6.7 [5.3,8.2]	1.8 [1.3,2.3]	1.1 [0.7,1.5]
Order effects (conditional on a partial match)				
n (errors)	146	160	291	388
First-k (prefix run)	28.7 [24.0,33.7]	41.3 [37.2,45.6]	40.1 [36.8,43.2]	37.6 [34.1,41.2]
Last-k (suffix run)	27.9 [22.4,33.0]	19.7 [15.3,23.6]	25.4 [22.2,28.6]	26.2 [22.9,29.6]
Contiguous (interior)	3.8 [1.9,6.4]	2.3 [0.9,4.1]	1.3 [0.4,2.3]	1.7 [0.7,3.0]
Other	39.3 [34.0,45.0]	36.5 [32.0,41.1]	33.2 [29.6,36.7]	34.4 [30.4,38.0]

Table S8 Error categories by model configuration (across all tasks with errors). Attempt-weighted population marginals from the hierarchical categorical model (§3.8.3); posterior median and 94% HDI, in percent. Error categories are the seven cognitive types of §1.4 and *unclassifiable* (†) is the catch-all category.

	Qwen3.5-27B	**GPT-OSS-120B**		
		high	**mid**	**low**
n (errors)	197	236	713	989
Attribute misselection	61.0 [58.7,63.5]	58.4 [55.9,60.9]	45.7 [44.3,47.3]	38.0 [36.6,39.4]
Transformation misselection	13.0 [11.4,14.5]	12.6 [11.1,14.3]	11.0 [10.0,12.1]	10.1 [9.3,10.9]
Partial program	7.2 [5.4,8.8]	10.2 [8.2,12.2]	10.8 [9.6,12.0]	9.5 [8.4,10.6]
Scope restriction	3.2 [2.3,4.3]	3.9 [3.0,4.7]	2.7 [2.4,3.1]	3.0 [2.5,3.6]
Operator confusion	0.9 [0.1,2.3]	3.2 [2.0,4.7]	2.7 [1.9,3.5]	3.6 [2.7,4.5]
Boundary precision	3.4 [2.4,4.4]	1.3 [0.6,2.0]	1.8 [1.3,2.3]	1.5 [1.1,1.8]
Sequencing	1.6 [0.8,2.3]	0.5 [0.0,1.1]	2.3 [1.8,2.9]	0.9 [0.5,1.3]
Unclassifiable †	9.6 [8.2,10.9]	9.7 [8.3,11.2]	22.8 [21.8,24.1]	33.4 [32.3,34.5]

Table S9 Human-versus-LLM source prediction by feature set and classifier (§3.7). Out-of-fold performance on the 965 errors (264 human, 701 LLM) from the 60 matched tasks, for classifiers trained on errors from all tasks. LLM errors are the positive class and F1 uses a probability threshold of 0.5. *Pooled* metrics are computed over all errors; *task-averaged* metrics are computed within each task and averaged with equal task weight. Point estimates with 95% percentile confidence intervals from 10,000 bootstrap resamples of whole tasks, shared across all rows.

	Pooled		**Task-averaged**	
	ROC-AUC	**F1**	**ROC-AUC**	**F1**
Primitive deltas ($\mathbf{v}_\Delta$)				
L2 logistic regression	0.652 [0.598,0.704]	0.708 [0.639,0.767]	0.706 [0.647,0.763]	0.624 [0.545,0.700]
EBM	0.652 [0.599,0.704]	0.747 [0.694,0.795]	0.677 [0.620,0.732]	0.686 [0.621,0.746]
Hist. gradient boosting	0.683 [0.621,0.739]	0.748 [0.692,0.798]	0.658 [0.595,0.718]	0.691 [0.631,0.748]
Category transitions ($\mathbf{v}_\rightarrow$)				
L2 logistic regression	0.611 [0.561,0.662]	0.649 [0.593,0.698]	0.616 [0.555,0.678]	0.589 [0.524,0.652]
EBM	0.606 [0.554,0.657]	0.688 [0.644,0.729]	0.594 [0.531,0.658]	0.651 [0.601,0.697]
Hist. gradient boosting	0.589 [0.530,0.646]	0.647 [0.596,0.692]	0.581 [0.517,0.643]	0.593 [0.537,0.648]
Concatenated ($\mathbf{v}_\Delta$, $\mathbf{v}_\rightarrow$)				
L2 logistic regression	0.675 [0.626,0.723]	0.739 [0.677,0.792]	0.674 [0.613,0.735]	0.657 [0.581,0.730]
EBM	0.677 [0.625,0.728]	0.746 [0.697,0.790]	0.671 [0.601,0.736]	0.692 [0.633,0.749]
Hist. gradient boosting	0.682 [0.621,0.740]	0.751 [0.695,0.799]	0.686 [0.623,0.747]	0.683 [0.619,0.743]

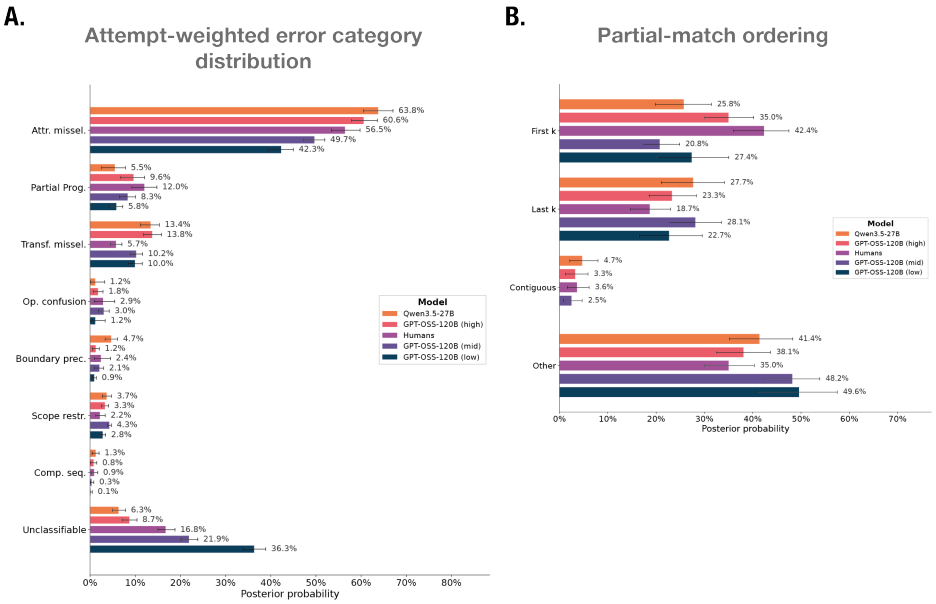


Fig. S4 Language models display human-like error category profiles, but order effects are weaker and less consistent than in humans. All panels report values corresponding to the subset of 60 tasks where evaluated LLMs and people both generated at least one incorrect response. (**A**) Prevalence of each error category among modeled incorrect machine errors. As with human results, an LLM classifier assigned every program one of seven cognitive categories or *unclassifiable*, conditioned on the reference program, the task grids, the synthesized program and its execution trace (Table 1 for definitions). Bars give the posterior median of the attempt-weighted prevalence derived from a hierarchical Bayesian categorical model, and error bars the 94% HDI. The categories are mutually exclusive. (**B**) Conditional on an attempt being explained by a partial-match program, we show the positions of the training examples it reproduces: the first-k or last-k example(s) shown, a contiguous block of at least two examples that does not include the first or last examples, or any other configuration. In both plots, bars give the posterior median attempt-weighted prevalence and error bars the 94% HDI.

Appendix S6 Prompts

Supplementary Figures S5–S8 reproduce the three prompt templates used by the pipeline. Placeholders in double braces are substituted at run time with the corresponding task or program text.

Error synthesis prompt (modify-only) `prompts/synthesize_error_program_modify_only.md`

You are an expert computational cognitive scientist and program synthesis researcher. Participants were given visual program synthesis tasks, and you are trying to model their reasoning. The participant saw a small set of training examples before producing their test output. Your task is to modify a few lines of an existing program to reverse-engineer approximate rule representations.

Task Overview

You will be given: (1) the full DSL for writing programs that solve visual program synthesis tasks; (2) a ground-truth program written in the DSL that solves the specific task a human participant tried; (3) the training and test examples for the visual program synthesis task; (4) participant data including the human's test output grid.
Your goal is to modify the ground-truth program such that when executed, it produces an output that closely matches the human's response. Since you are limited in how much you can modify the program, the resulting modified program should capture the core conceptual aspects of the human's thinking as much as possible. A program capturing the participant's general strategy is preferable to one that perfectly reconstructs every cell of their output. The question is what alternative or partial rule the participant may have inferred from the training examples, and how that rule diverges from the ground-truth program.

DSL Reference

The following DSL is available for program modification:

`{{dsl_code}}`	*DSL signatures + docstrings*

Task data

`{{ref_program}}`	***Formal DSL Program*** *— ground-truth solver body*
`{{ref_examples}}`	***Training Examples (Input → Output)*** *— rendered train grid pairs*

Participant data

`{{participant_description}}`	*optional NL description written by the participant*
`{{participant_output}}`	***Participant's Output*** *— the human's incorrect test grid*

Instructions

Produce an alternative program to the ground-truth program that can reproduce the error made by the human participant. You are only allowed to modify `{{num_modified_lines}}` lines of code in the program, so choose them carefully. The lines of code you choose to modify can be rewritten in the following ways:

1. Modifying an argument to a function (e.g. `replace(I, SIX, TWO)` → `replace(I, SIX, THREE)`)
2. Replacing an existing function with another function (e.g. `vmirror(x1)` → `hmirror(x1)`)
3. Regenerating a line of code completely using the DSL (e.g. `upscale(I, THREE)` → `rot90(I)`)

Here are some rules for writing each line of code:

Each line you modify or regenerate must follow the same syntax as the ground-truth program: `x[k] = function_name(arg1, ..., argL)`, where arguments are only variable names from the existing program, DSL constants, or DSL function names passed by reference (e.g. `mapply(outbox, x2)`). No nested calls (`f(g(x))`). You cannot use any operations other than those in the DSL: no `def`, no lambda expressions, no Python primitive operators (`*`, `//`, `+`, `-` — use the DSL equivalents), no native Python lists/tuples/dicts/sets, no double assignment or `*`-unpacking, and no direct assignment of numbers (use constants).

[nine bullets in the source, collapsed here; wording otherwise unchanged]

Your outputs will be used to replace each of the lines you choose to modify in the original program. The resulting program should produce an output that is consistent with the participant's response on the test input.

Response Format

Respond **only** with a JSON object of this form:

```
{
    "modifications": [
       { "variable_name": "<var>", "replacement": "<rhs>" }
    ]
}
```

- **variable_name**: Exact variable from the ground-truth program whose assignment line to change (e.g. `"x1"`, `"x2"`, `"O"`).
- **replacement**: The new right-hand side only (e.g. `"rotate(I)"`, `"objects(I, T, F, T)"`). Same DSL syntax: `function_name(arg1, arg2, ...)`. No `var =` prefix.
- **modifications**: At most `{{num_modified_lines}}` entries. Each entry replaces one line: the line `variable_name = ...` becomes `variable_name = replacement`.

Example: to change `x3 = hmirror(x2)` to `x3 = vmirror(x2)`, output `{"variable_name": "x3", "replacement": "vmirror(x2)"}` inside `modifications`.

Fig. S5 Error-program synthesis prompt (§3.3). The model is given the full DSL specification, the task grids, the reference program implementing the target rule, and the respondent's incorrect output grid, and returns a structured set of line edits under a fixed JSON schema.

Solver synthesis prompt `prompts/synthesize_solver_program.md`

You are an expert program synthesis researcher. Your task is to write a program in a domain-specific language (DSL) that solves a visual pattern transformation task. The task consists of input–output grid pairs that demonstrate a consistent transformation rule. You must discover this rule and express it as a DSL program.

DSL Reference

The following DSL is available for writing programs:

`{{dsl_code}}` — *DSL signatures + docstrings*

`{{in_context_examples}}` — *retrieved demo solvers; empty under zero-shot*

Target Task

Write a solver program for the following task. Study the training examples to discover the transformation rule, then express it as a DSL program.

`{{target_examples}}` — *train pairs, test input; test output hidden by default*

`{{task_description}}` — *optional human NL description of the rule*

Instructions

Write a DSL program that transforms the input grid to the output grid for the target task above. The program should generalize the transformation rule demonstrated by the training examples.

Syntax rules:

- Each line of code must follow this form: `x[k] = function_name(arg1, ..., argL)` for line k and L arguments.
- The final output variable must be named `O`.
- Arguments can only be: variable names from previous lines in the program, constants from the DSL, or function names from the DSL passed as references (e.g. `mapply(outbox, x2)` where `outbox` is passed by name, not called).
- Do not nest function calls — that is, do not use a function call as an argument to another function (e.g. `f(g(x))` is not allowed).
- You cannot use any other operations than the ones in the DSL.
  - Do not use nested functions like a `def` or nested function calls like `func(func(x))`. Instead, compose functions using existing DSL functions.
  - Do not use lambda expressions.
  - Do not use Python primitive operations like `*`, `//`, `+`, `-` and others. For those operations, use the corresponding DSL functions, when available.
  - Do not directly create native Python data structures like lists, tuples, dictionaries, sets.
  - Do not use double variable assignments or use of `*` operator for arguments.
  - Do not perform direct assignment of numbers. Instead, use constants where appropriate.
- The input grid is always available as the variable `I`.

Response Format

Respond **only** with a JSON object of this form:

```
{
   "program": "x1 = ...\nx2 = ...\nO = ..."
}
```

- **program**: The complete solver program body, with lines separated by `\n`. Each line assigns to a variable. The last meaningful assignment should produce the output grid, assigned to `O`.

Fig. S6 Reference-program (solver) synthesis prompt (§3.2.1). Used to generate reference programs for `evaluation`-split tasks, which have no hand-written solver. The prompt is retrieval-augmented with five solved tasks and their human-written solvers from the `training` split.

Error classification prompt (abbreviated) `prompts/classify_errors.md`

You are an expert computational cognitive scientist studying human errors on visual program synthesis tasks (ARC — Abstraction and Reasoning Corpus). For each participant error, we have synthesized a program that reproduces the participant's incorrect output. Your task is to characterize the cognitive error that this program represents using a structured taxonomy.

DSL Reference

`{{dsl_code}}` *signatures + docstrings*

Task Data

`{{ref_program}}` *ground-truth program, solves the task correctly*

`{{ref_examples}}` *training examples, Input → Output*

Error Program

`{{error_program}}` *synthesized to reproduce the participant's incorrect output*

`{{training_comparison}}` *side-by-side ground-truth vs. error-program outputs, per training example*

`{{participant_output}}` *error program's output on the test input*

`{{participant_description}}` *optional participant NL*

[one framing sentence per input section omitted, plus the two sub-headings under Error Program]

Error Category Taxonomy

Classify the error into exactly one of the following 8 categories. Read all definitions before deciding.

1. **Attribute Substitution** — replaces a quantity or feature that must be computed with a different, often simpler or hardcoded one.
2. **Scope Restriction** — the correct computation, applied correctly, but only to a subset of the targets it should cover.
3. **Operator Confusion** — right class of operation, wrong member of it (rotating left instead of right, mirroring on the wrong axis).
4. **Transformation Substitution** — right targets and spatial structure, but an operation from an entirely different family.
5. **Compositional Sequencing Error** — all the right operations, wrongly ordered (`sequential_ordering`) or wrongly bound to targets (`parallel_binding`).
6. **Partial Program** — the pipeline is incomplete; the steps present are correct, some are missing entirely.
7. **Boundary Precision Error** — correct quantity, off by one (or a small constant), not explainable by overfitting to training examples.
8. **Unclassifiable / Ad-hoc Rule** — an idiosyncratic rule bearing little structural relationship to the ground truth.

[each Definition abridged to one clause; the per-category Key diagnostic and Distinguishing criteria paragraphs (8 × 2) are omitted]

Quick Decision Guide

Ask these questions in order. Stop at the first "yes":

1. Is the participant's program a proper subset of the ground-truth pipeline, with missing steps representing unperformed computation? → **partial_program**
2. Is the correct operation applied, but not everywhere it should be? → **scope_restriction**
3. Is a different attribute/feature (e.g. colour, dimension) used in place of the computed/inferred one, perhaps resorting to a simpler strategy or hardcoded value? → **attribute_substitution**

   Sub-check: if the difference is only ± 1, do the training examples discriminate between the two values? If no → attribute_substitution (hardcoded_constant). If yes → boundary_precision (skip to 7).
4. Is the right operation class identified, but the wrong member of that class selected (e.g. wrong rotation direction, wrong mirroring axis)? → **operator_confusion**
5. Are the correct targets identified, but is a transformation from an entirely different computational family applied? → **transformation_substitution**
6. Are all the right operations identified and applied to all targets, but assembled in the wrong order or bound to the wrong targets? → **compositional_sequencing**
7. Is the computation off by exactly ± 1 from the correct value, and do the training examples rule out overfitting? → **boundary_precision**
8. Does the participant's program bear little structural resemblance to the ground truth? → **unclassifiable**

Response Instructions

Follow these three steps in order:

- **Step 1 — Rule descriptions.** `correct_rule_description` and `error_rule_description`: what the ground-truth program and the error program each do, as a rule a human could articulate. Cognitive/perceptual terms, never DSL ones; describe *behavior*, not *syntax*; translate colour codes to colour names.
- **Step 1b — Operation domain.** Name the mental operation at stake ("spatial placement", "object counting", "colour attribution"), focusing on what the participant failed at.
- **Step 2 — Category selection.** Walk through the Quick Decision Guide above; select the single best-fitting `error_category`.
- **Step 3 — Category-specific fields.** Fill in ONLY the fields for that category. All fields for other categories must be null.

[vocabulary-guidance paragraph omitted]

[the source's Field-to-Category Mapping table — 8 rows naming the 2–5 non-null fields required for each category — is omitted here for space; the fields are listed in `src/category_definitions.py`]

Response Format

Respond **only** with a JSON object containing all fields below; each field carries a one-line description of what to write there. Fields not belonging to the selected category must be null.

```
{
   "correct_rule_description": ..., "error_rule_description": ...,
   "operation_domain": ..., "error_category": <one of the 8 categories above>,
   <28 category-specific fields, non-null only for the selected category>
}
```

[≈30 category-specific fields follow in the source, each with a one-line description and an explicit `null` option]

Fig. S7 Error-classification prompt (§3.4). The classifier sees the reference program, the task grids, the synthesized error program, and a side-by-side comparison of their behavior on the training examples, and returns one of the seven cognitive categories or the unclassifiable catch-all.

LLM evaluation prompt — `prompts/llm_evaluation.md`

System message

```
You are a helpful assistant that solves analogy
making puzzles. Only give the answer, no other
words or text.
```

User message

```
Let's try to complete the pattern:

{examples}

input {query_number}: {test_input}

output {query_number}:
```

- `{examples}` — the task's training input/output grid pairs, numbered and rendered in the same `input n:` / `output n:` form as the query.
- `{query_number}` — the index of the held-out pair, continuing the numbering of `{examples}`.
- `{test_input}` — the test input grid. The matching output is left dangling after the final colon, so the model continues the pattern rather than answering a question.

Fig. S8 LLM evaluation prompt (§3.6). Used to collect machine responses on ARC tasks for comparison with the human data. The model is shown the training pairs and asked to complete the held-out pair.

References

[1] Shepard, R. N., Hovland, C. I. & Jenkins, H. M. Learning and memorization of classifications. **75**, 1–42. URL http://doi.apa.org/getdoi.cfm?doi=10.1037/h0093825.

[2] Fodor, J. A. *The language of thought* Vol. 5 (Harvard university press, 1975).

[3] Newell, A. Physical symbol systems **4**, 135–183. URL https://onlinelibrary.wiley.com/doi/10.1207/s15516709cog0402_2.

[4] Braine, M. D. & O'Brien, D. P. *Mental Logic* (Psychology Press).

[5] Goodman, N. D., Tenenbaum, J. B., Feldman, J. & Griffiths, T. L. A rational analysis of rule-based concept learning. *Cognitive science* **32**, 108–154 (2008).

[6] Sloman, S. & Sloman, S. A. *Causal Models: How People Think about the World and Its Alternatives* (Oxford University Press).

[7] Tenenbaum, J. B., Kemp, C., Griffiths, T. L. & Goodman, N. D. How to Grow a Mind: Statistics, Structure, and Abstraction. *Science* **331**, 1279–1285 (2011).

[8] Lake, B. M., Salakhutdinov, R. & Tenenbaum, J. B. Human-level concept learning through probabilistic program induction. *Science* **350**, 1332–1338 (2015).

[9] Yang, Y. & Piantadosi, S. T. One model for the learning of language **119**, e2021865119. URL https://www.pnas.org/doi/abs/10.1073/pnas.2021865119.

[10] Rule, J. S. *et al.* Symbolic metaprogram search improves learning efficiency and explains rule learning in humans **15**, 6847. URL https://www.nature.com/articles/s41467-024-50966-x.

[11] Brown, T. B. *et al.* Language Models are Few-Shot Learners URL http://arxiv.org/abs/2005.14165.

[12] Wei, J. *et al.* Emergent Abilities of Large Language Models. URL http://arxiv.org/abs/2206.07682. 2206.07682.

[13] Guo, D. *et al.* DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning **645**, 633–638. URL https://www.nature.com/articles/s41586-025-09422-z.

[14] Lake, B. M. & Baroni, M. Human-like systematic generalization through a meta-learning neural network 1–7. URL https://www.nature.com/articles/s41586-023-06668-3.

[15] Webb, T., Holyoak, K. J. & Lu, H. Emergent analogical reasoning in large language models **7**, 1526–1541. URL https://www.nature.com/articles/s41562-023-01659-w.

[16] Binz, M. *et al.* Meta-learned models of cognition. *Behavioral and Brain Sciences* **47**, e147 (2024).

[17] Musker, S., Duchnowski, A., Millière, R. & Pavlick, E. LLMs as models for analogical reasoning **145**, 104676. URL https://www.sciencedirect.com/science/article/pii/S0749596X25000695.

[18] Binz, M. *et al.* A foundation model to predict and capture human cognition **644**, 1002–1009. URL https://www.nature.com/articles/s41586-025-09215-4.

[19] Jagadish, A. K., Thalmann, M., Coda-Forno, J., Binz, M. & Schulz, E. Meta-learning ecological priors from large language models explains human learning and decision making. *arXiv preprint arXiv:2509.00116* (2025).

[20] Loo, A., Pavlick, E. & Feiman, R. LLMs model how humans induce logically structured rules **146**, 104675. URL https://www.sciencedirect.com/science/article/pii/S0749596X25000683.

[21] Griffiths, T. L., Lake, B. M., McCoy, R. T., Pavlick, E. & Webb, T. W. Whither symbols in the era of advanced neural networks? **0**. URL https://www.cell.com/

trends/cognitive-sciences/abstract/S1364-6613(26)00028-8.

[22] McClelland, J. L. *et al.* Letting structure emerge: Connectionist and dynamical systems approaches to cognition **14**, 348–356. URL https://linkinghub.elsevier.com/retrieve/pii/S1364661310001245.

[23] Lake, B. M. & Piantadosi, S. T. People Infer Recursive Visual Concepts from Just a Few Examples **3**, 54–65. URL http://link.springer.com/10.1007/s42113-019-00053-y.

[24] Ellis, K. *et al. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning* (ACM). URL https://dl.acm.org/doi/10.1145/3453483.3454080.

[25] Sablé-Meyer, M., Ellis, K., Tenenbaum, J. & Dehaene, S. A language of thought for the mental representation of geometric shapes **139**, 101527. URL https://www.sciencedirect.com/science/article/pii/S0010028522000639.

[26] Zhou, Y., Feinman, R. & Lake, B. M. Compositional diversity in visual concept learning **244**, 105711. URL https://www.sciencedirect.com/science/article/abs/pii/S0010027723003451.

[27] Forbes, A. R. An Item Analysis of the Advanced Matrices. *British Journal of Educational Psychology* **34**, 223–236 (1964).

[28] Brown, J. S. & Burton, R. R. Diagnostic models for procedural bugs in basic mathematical skills **2**, 155–192. URL https://www.sciencedirect.com/science/article/pii/S0364021378800044.

[29] VanLehn, K. *Mind Bugs: The Origins of Procedural Misconceptions* Mind Bugs: The Origins of Procedural Misconceptions (The MIT Press).

[30] Kunda, M., Soulières, I., Rozga, A. & Goel, A. K. Error patterns on the Raven's Standard Progressive Matrices Test **59**, 181–198. URL https://www.sciencedirect.com/science/article/pii/S0160289616300149.

[31] Chen, M. *et al.* Evaluating Large Language Models Trained on Code. URL http://arxiv.org/abs/2107.03374. 2107.03374.

[32] Austin, J. *et al.* Program Synthesis with Large Language Models. URL http://arxiv.org/abs/2108.07732. 2108.07732.

[33] DeepSeek-AI *et al.* DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. URL http://arxiv.org/abs/2406.11931. 2406.11931.

[34] Wong, L. *et al.* Modeling open-world cognition as on-demand synthesis of probabilistic models. *arXiv preprint arXiv:2507.12547* (2025).

[35] Rmus, M., Jagadish, A. K., Mathony, M., Ludwig, T. & Schulz, E. Generating Computational Cognitive Models using Large Language Models. URL http://arxiv.org/abs/2502.00879. 2502.00879.

[36] Castro, P. S. *et al.* Discovering Symbolic Cognitive Models from Human and Animal Behavior. URL https://www.biorxiv.org/content/10.1101/2025.02.05.636732v1.

[37] Chandramouli, S., Kachergis, G. & Jagadish, A. Automated adversarial collaboration for advancing theory building in the cognitive sciences. *arXiv preprint arXiv:2604.25521* (2026).

[38] Xie, H., Jagadish, A. K., Pan, L. & Wilson, R. C. Think-aloud reshapes automated cognitive model discovery beyond behavior. *arXiv preprint arXiv:2605.05091* (2026).

[39] Jagadish, A. K. *et al.* Closing the Loop to Discover Psychological Theories with an Automated Cognitive Scientist. URL http://arxiv.org/abs/2606.26448. 2606.26448.

[40] Prystawski, B. *et al.* Auto-psych: Automating the science of mind using agent-driven theory discovery and experimentation. URL http://arxiv.org/abs/2606.26460. 2606.26460.

[41] Chater, N. & Vitányi, P. Simplicity: A unifying principle in cognitive science? **7**, 19–22. URL https://linkinghub.elsevier.com/retrieve/pii/S1364661302000050.

[42] Feldman, J. The Simplicity Principle in Human Concept Learning **12**, 227–232. URL https://doi.org/10.1046/j.0963-7214.2003.01267.x.

[43] Simon, H. A. Rational choice and the structure of the environment. *Psychological Review* **63**, 129–138 (1956).

[44] Kahneman, D. & Frederick, S. in *Representativeness Revisited: Attribute Substitution in Intuitive Judgment Heuristics and Biases: The Psychology of Intuitive Judgment* (eds Griffin, D., Kahneman, D. & Gilovich, T.) 49–81 (Cambridge University Press, 2002).

[45] Chollet, F. On the measure of intelligence (2019). URL https://arxiv.org/abs/1911.01547. arXiv:1911.01547.

[46] Hodel, M. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation (2024). arXiv:2404.07353.

[47] LeGris, S., Vong, W. K., Lake, B. M. & Gureckis, T. M. A Comprehensive Behavioral Dataset for the Abstraction and Reasoning Corpus. *Scientific Data* **12**, 1380 (2025).

[48] Kendall, C. & Oprea, R. On the complexity of forming mental models **15**, 175–211. URL https://www.econometricsociety.org/doi/10.3982/QE2264.

[49] Simon, H. A. *Models of Bounded Rationality* (MIT Press, 1982).

[50] Qwen Team. Qwen3.5: Towards native multimodal agents (2026). URL https://qwen.ai/blog?id=qwen3.5.

[51] OpenAI. gpt-oss-120b & gpt-oss-20b model card (2025). URL https://arxiv.org/abs/2508.10925. arXiv:2508.10925.

[52] LeGris, S., Vong, W. K., Lake, B. M. & Gureckis, T. M. H-ARC: A robust estimate of human performance on the Abstraction and Reasoning Corpus benchmark. URL http://arxiv.org/abs/2409.01374.

[53] Allen, K. *et al.* Using games to understand the mind **8**, 1035–1043. URL https://www.nature.com/articles/s41562-024-01878-9.

[54] Newell, A. *Unified Theories of Cognition* (Harvard University Press, Cambridge, MA, 1990).

[55] Newell, A. & Simon, H. A. *Human Problem Solving* (Prentice-Hall, Englewood Cliffs, NJ, 1972).

[56] de Varda, A. G., D'Elia, F. P., Kean, H., Lampinen, A. & Fedorenko, E. The cost of thinking is similar between large reasoning models and humans **122**, e2520077122. URL https://www.pnas.org/doi/abs/10.1073/pnas.2520077122.

[57] Lieder, F. & Griffiths, T. L. Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources **43**, e1. URL https://www.cambridge.org/core/journals/behavioral-and-brain-sciences/article/resourcerational-analysis-understanding-human-cognition-as-the-optimal-use-of-limited-computational-resou
586866D9AD1D1EA7A1EECE217D392F4A.

[58] Khemlani, S. & Johnson-Laird, P. N. Theories of the syllogism: A meta-analysis. *Psychological Bulletin* **138**, 427–457 (2012).

[59] Marcus, G. F. *et al.* Overregularization in Language Acquisition. *Monographs of the Society for Research in Child Development* **57**, i–178 (1992).

[60] Sanborn, A. N., Griffiths, T. L. & Navarro, D. J. Rational approximations to rational models: Alternative algorithms for category learning **117**, 1144–1167.

[61] Chomsky, N. *Aspects of the Theory of Syntax* 50 edn (The MIT Press). URL https://www.jstor.org/stable/j.ctt17kk81z. j.ctt17kk81z.

[62] Marr, D. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information* (Henry Holt and Co., Inc.).

[63] Anderson, J. R. *The Adaptive Character of Thought* The Adaptive Character of Thought (Lawrence Erlbaum Associates, Inc).

[64] Babcock, R. L. Analysis of age differences in types of errors on the Raven's Advanced Progressive Matrices. *Intelligence* **30**, 485–503 (2002).

[65] Ellis, K., Morales, L. & Sablé-Meyer, M. Library Learning for Neurally-Guided Bayesian Program Induction URL https://papers.nips.cc/paper_files/paper/2018/file/7aa685b3b1dc1d6780bf36f7340078c9-Paper.pdf.

[66] Kovacic, Z., Chiu, J. T., Lee, C., Zhao, W. & Ellis, K. Refactoring Codebases through Library Design. URL http://arxiv.org/abs/2506.11058. 2506.11058.

[67] Wong, C., Ellis, K., Tenenbaum, J. B. & Andreas, J. Leveraging language to learn program abstractions and search heuristics. URL http://arxiv.org/abs/2106.11053.

[68] Acquaviva, S. *et al.* Communicating Natural Programs to Humans and Machines. URL http://arxiv.org/abs/2106.07824.

[69] Novikov, A. *et al.* AlphaEvolve: A coding agent for scientific and algorithmic discovery .

[70] Fawzi, A. *et al.* Discovering faster matrix multiplication algorithms with reinforcement learning **610**, 47–53. URL https://www.nature.com/articles/s41586-022-05172-4.

[71] Jansen, P. *et al.* CodeScientist: End-to-End Semi-Automated Scientific Discovery with Code-based Experimentation. URL http://arxiv.org/abs/2503.22708. 2503.22708.

[72] Ghareeb, A. E. *et al.* A multi-agent system for automating scientific discovery **655**, 497–505. URL https://www.nature.com/articles/s41586-026-10652-y.

[73] Romera-Paredes, B. *et al.* Mathematical discoveries from program search with large language models **625**, 468–475. URL https://www.nature.com/articles/s41586-023-06924-6.

[74] Wurgaft, D. *et al.* Scaling up the think-aloud method. URL http://arxiv.org/abs/2505.23931. 2505.23931.

[75] Wang, C., Kasenberg, D., Stachenfeld, K. & Castro, P. S. Discovering Differences in Strategic Behavior Between Humans and LLMs. URL http://arxiv.org/abs/2602.10324. 2602.10324.

[76] Waterman, D. & Newell, A. Protocol analysis as a task for artificial intelligence **2**, 285–318. URL https://linkinghub.elsevier.com/retrieve/pii/0004370271900142.

[77] Sabry, A. & Felleisen, M. Reasoning about programs in continuation-passing style. *SIGPLAN Lisp Pointers* **V**, 288–298 (1992). URL https://doi.org/10.1145/141478.141563.

[78] Lewis, P. *et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems*, Vol. 33, 9459–9474 (Curran Associates, Inc.). URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[79] Butt, N. *et al. Codeit: Self-improving language models with prioritized hindsight replay. International Conference on Machine Learning* (PMLR, 2024). URL https://arxiv.org/abs/2402.04858. 2402.04858.

[80] Hoffman, M. D., Gelman, A. *et al.* The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo. *J. Mach. Learn. Res.* **15**, 1593–1623 (2014).

[81] Phan, D., Pradhan, N. & Jankowiak, M. Composable effects for flexible and accelerated probabilistic programming in numpyro. *arXiv preprint arXiv:1912.11554* (2019).

[82] Mc McElreath, R. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan* 2nd edn (Chapman and Hall/CRC, Boca Raton, FL, 2020). URL https://www.taylorfrancis.com/books/mono/10.1201/9780429029608/statistical-rethinking-richard-mcelreath.